10/12/2026
LRG
10/12/2026

Lab 8 - UART and Serial Device Interface

• Purpose: Build a bidirectional serial interface for the PIC16F883, then integrate UART communication with ADC measurement and precise servo timing. The Lab progresses from one repeating byte to byte echo, structured command packets, LM34 temperature measurements, and simultaneous UART/ADC/servo operation.

- Repository setup: Create a Git repository for Lab 8 before starting projects. Open the student lab .gitignore template, select Raw, and save the file as .gitignore in the repository root along side README.md. Do this before first git add so generated files are excluded. Keep the PIC firmware and host application in separate directions as shown in the assignment repository structure.

• Equipment and Materials:

- oscilloscope with serial decode
- frequency counter
- digital multimeter
- suitable servo power supply
- independent temperature refrenence.
- PICkit 3
- Computer with MPLAB X, pic-as, a serial terminal, and host program capable of sending/receiving binary bytes
- USB-to-RS-232 adapter
- PIC16F883 circuit with 4MHz crystal.
- MAX232 or equivalent RS-232 transceiver and required capacitors.
- potentiometer
- LM34 temp sensor
- one servo for Part 4
- Lab book and various components.

• Part 1 - UART TX and RS-232 Proof of Life:

• Goal: Configure the PIC16F883 EUSART for 9600 baud, 8N1, and continuously transmit a selected byte. Predict and measure its bit pattern, timing, and voltage levels on both the PIC UART and RS-232 signals.

• UART frame & timing -

- 8N1 = (8 = eight data bits), (N = no parity bit), & (1 = one stop bit)

An asynchronous 8N1 character also has one start bit:

1 start + 8 data + 0 parity + 1 stop = 10 bit frames

For a target baud rate of 9600 baud:

$$\cdot T_{bit} = \frac{1}{9600} = \underline{104.17\mu s}$$

One complete 8N1 byte takes:

$$T_{byte} = 10\, T_{bit}$$

$$10 \cdot 104.17\mu s$$

$$T_{byte} = 1.0417ms$$

Use the baud rate generator settings you selected to calculated the expected actual baud rate and error. Measure actual bit time on the oscilloscope and calculate measure baud rate

$$\text{baud measured} = \frac{1}{T_{bit\ measured}}$$

10/12/2026
LRG

10/12/2026
LRG

Lab 8 - UART

- Worked example: baud rate generate and error at 9600 baud:

What?: = calculate the baud rate generator setting, actual baud rate, and % error
Why?: = the integer baud rate setting causes the actual rate to differ slightly from the target.

$$n = \frac{Fosc}{16B_{target}} - 1 = \frac{4,000,000}{16(9600)} - 1 = \underline{25.0417}$$

Calculate the actual baud rate:

$$B_{actual} = \frac{FOSC}{16(n+1)} = \frac{4,000,000}{16(25+1)} = \underline{9615.38 \text{ baud.}}$$

Calculate the % error relative to the target:

$$Error = \frac{B_{actual} - B_{target}}{B_{target}} \times 100\% = \frac{9615.38 - 9600}{9600} \times 100\% = \underline{0.16\%}$$

- Required setup & Program structure: See RCET3375/LabAssignments/Lab8-UART for more information but steps were followed in program structure.

- Before Lab:
  - EUSART SFR documentation for TXSTA, RCSTA, BAUDCTL, SPBRG, TXREG, and relevant PIR1 flags.
  - baud rate generator setting, calculated rate, and percent error for 9600 baud at actual oscillator freq.
  - explanation of 8N1 and predicted bit/byte time
  - program flowchart and source.
  - predicted start, D0-D7 (LSB first), and stop levels for $ = 0x24 and contrasting test bytes
  - expected ideal idle/start polarity and voltages on both TTL & RS-232
  - MAX232/adapter schematic, electrical loading/component verification, and recap required capacitors and voltage ratings.
  - a test plan for each TTL to RS-232 driver and RS-232-to-TTL receiver channel

- In the Lab:

1) Build the MAX232 level converter circuit with the required charges capacitors, Verify the connections and components ratings.

2) Test every TTL to RS-232 drive channel and every RS-232 to TTL receiver channel independantly using appropriate signals. Measure the voltage levels and verify the polarity of each channel.

3) Configure asynchronous EUSART Transmission at 9600 baud, 8N1, and repeatedly transmit selected byte, starting (B)

4) You may add a delay between transmissions of approximately one byte time. Its exact duration is not critical.

5) Display RC6/TX (TTL) and the corresponding MAX232 RS-232 output on separate oscilloscope at the same time.

6) Identify the start bit, D0 through D7, and stop bit directly from waveforms. Compare the sequence with prediction.

7) Try other repeated bytes such as 0xFF, 0x00, 0x55, 0xAA, 0xF7, 0x08, or another useful test value. Record the predicted and observed bit sequence.

8) Choose the clearest pattern for a scope capture. Adjust the timebase so one complete 10 bit frame fills as much of the screen as possible, with both traces visible and the start/stop boundaries intact.

9) On that capture, identify start, D0-D7, and stop for both TTL and RS232. Mark the bit time clearly.

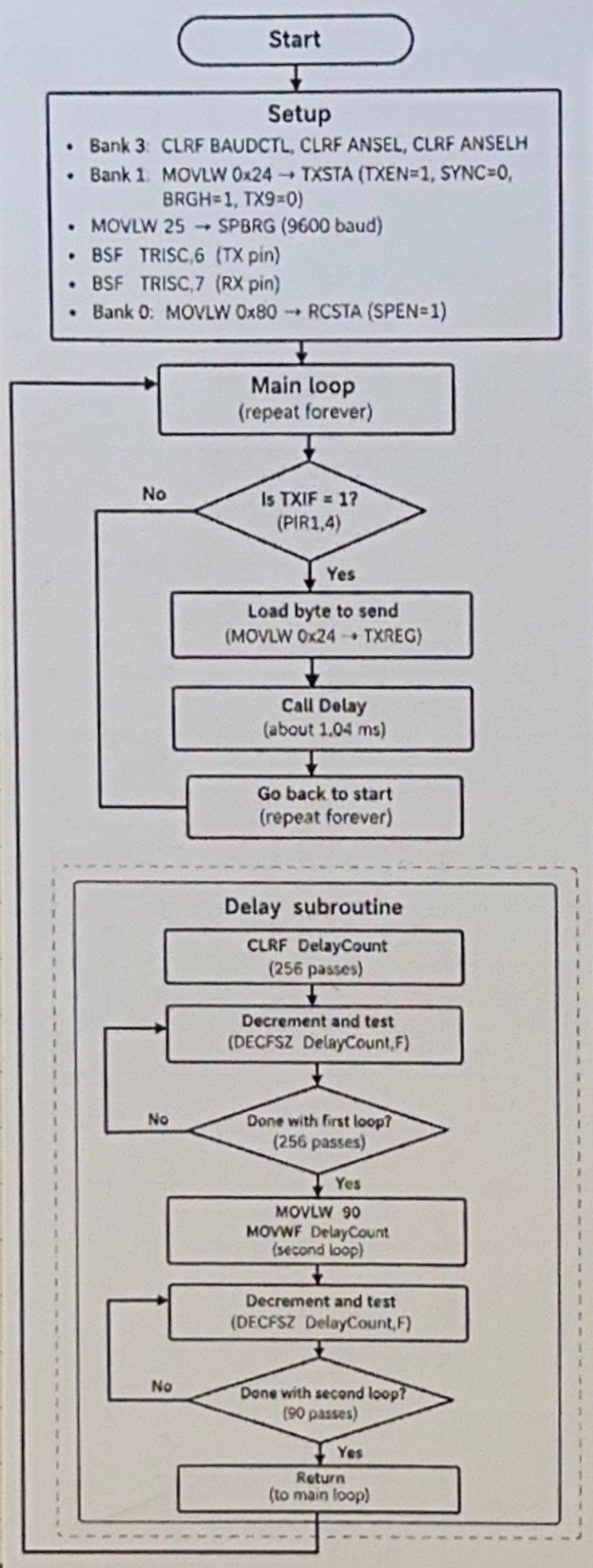
10) Measure bit time and complete frame time. Calculate measured baud rate and verify that the frame occupies 10 bit frame

11) Measure and record the TTL and RS-232 voltages for idle, start, and data levels. Explain the polarity change between the two signals.

10/12/2026
LRG

Lab 8- UART

L6



Setup (details)

1. ANSEL: RA0 analog (others digital)
2. ANSELH: clear
3. TXSTA: TXEN=1, SYNC=0, BRGH=1, TX9=0
4. SPBRG: 25 (9600 baud @ 4 MHz)
5. TRISC: RC6 (TX) and RC7 (RX) as outputs (datasheet recommendation)
6. RCSTA: SPEN=1 (enable serial port)
7. Interrupts: not used (vector traps)

Delay subroutine (details)

- First loop: 256 passes × 3 cycles − 1 = 767 cycles
- Second loop: 90 passes × 3 cycles − 1 = 269 cycles
- Plus setup, CALL and RETURN ≈ 7 cycles

Total ≈ 1,043 cycles

At 1 MHz instruction rate (1 µs/cycle):

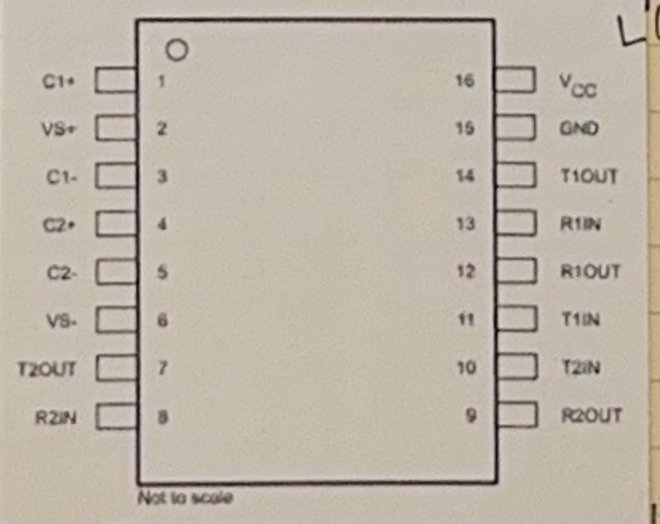
1,043 µs ≈ 1.04 ms

Key values / calculations

- Baud rate (BRGH=1, BRG16=0):
  n = 4,000,000 / (16 × 9600) − 1 = 25.04 → 25
- Actual baud: 4,000,000 / (16 × 26) = 9615.38
- Error: +0.16%
- Byte time (10 bits): 1 / 9600 = 104.17 µs
  (used for delay ≈ 1.04 ms)

•Figure 1: UART TX & RS-232 Proof of Life flow chart:

6



•Figure 2: MAX232 Pinout diagram

-The MAX222 has 2 drivers and 2 receivers. Input level voltage is ±30V. Each receivers converts TIA/EIA-232-F inputs to 5v TTL/CMOS levels. The receivers have a typical threshold of 1.3v, and typical hysteresis of 0.5v.

•Table 1: MAX232 Absolute Maximum Ratings

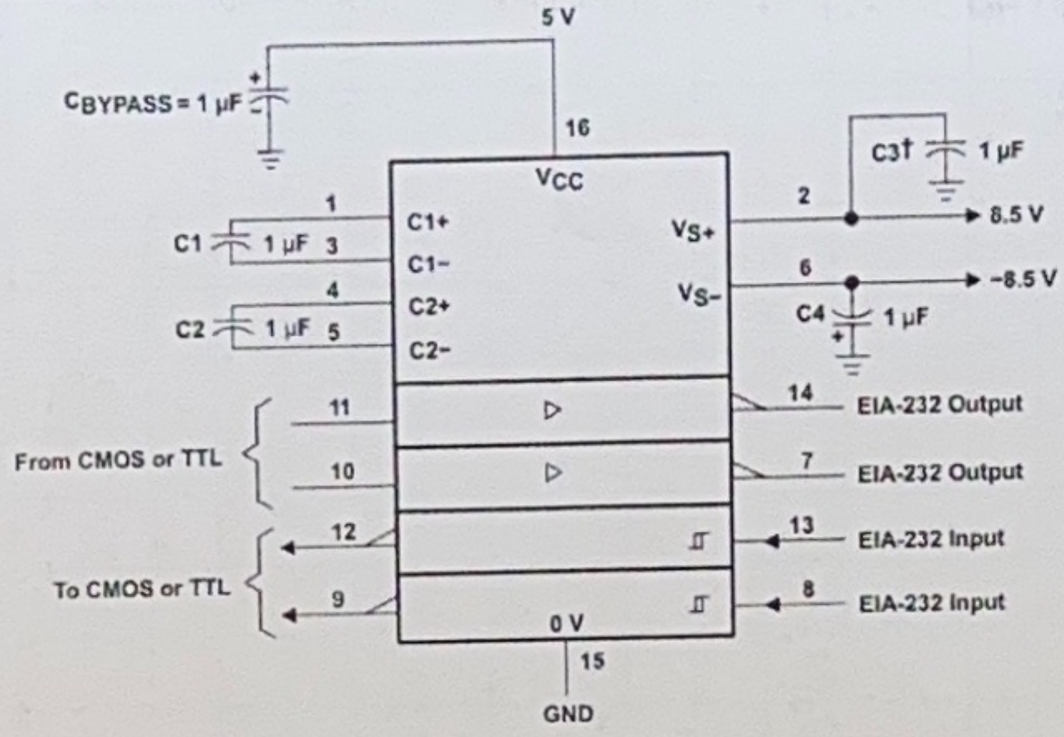
6

			MIN	MAX	UNIT
V_{CC}	Input Supply voltage range[2]		−0.3	6	V
V_{S+}	Positive output supply voltage range		V_{CC} − 0.3	15	V
V_{S-}	Negative output supply voltage range		−0.3	−15	V
V_I	Input voltage range	T1IN, T2IN	−0.3	V_{CC} + 0.3	V
		R1IN, R2IN		±30	
V_O	Output voltage range	T1OUT, T2OUT	V_{S-} − 0.3	V_{S+} + 0.3	V
		R1OUT, R2OUT	−0.3	V_{CC} + 0.3	
	Short-circuit duration	T1OUT, T2OUT	Unlimited		
T_J	Operating virtual junction temperature			150	°C

Lab8-UART

10/12/2026
LRG

LG



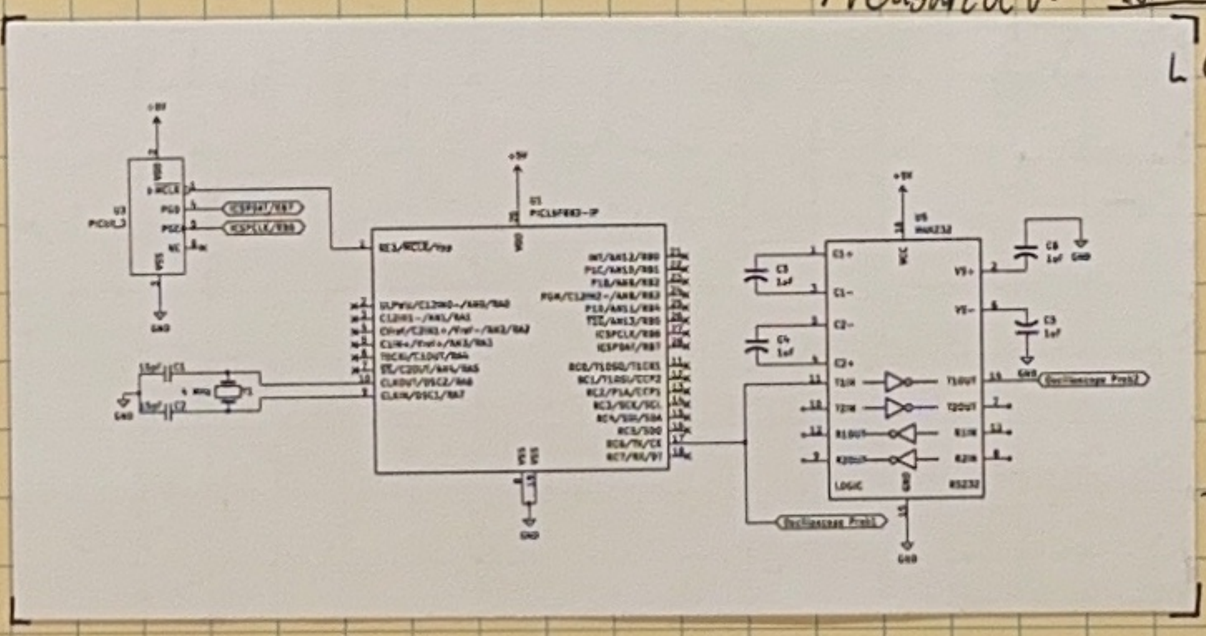
† C3 can be connected to V_{CC} or GND.

NOTES: A. Resistor values shown are nominal.
B. Nonpolarized ceramic capacitors are acceptable. If polarized tantalum or electrolytic capacitors are used, they should be connected as shown. In addition to the 1-µF capacitors shown, the MAX202 can operate with 0.1-µF capacitors.

•Figure 3: MAX 232 typical operating circuit

• Measured Vs+ = 8.86V
Measured Vs− = ~~-8.38V~~ -8.33V

LG



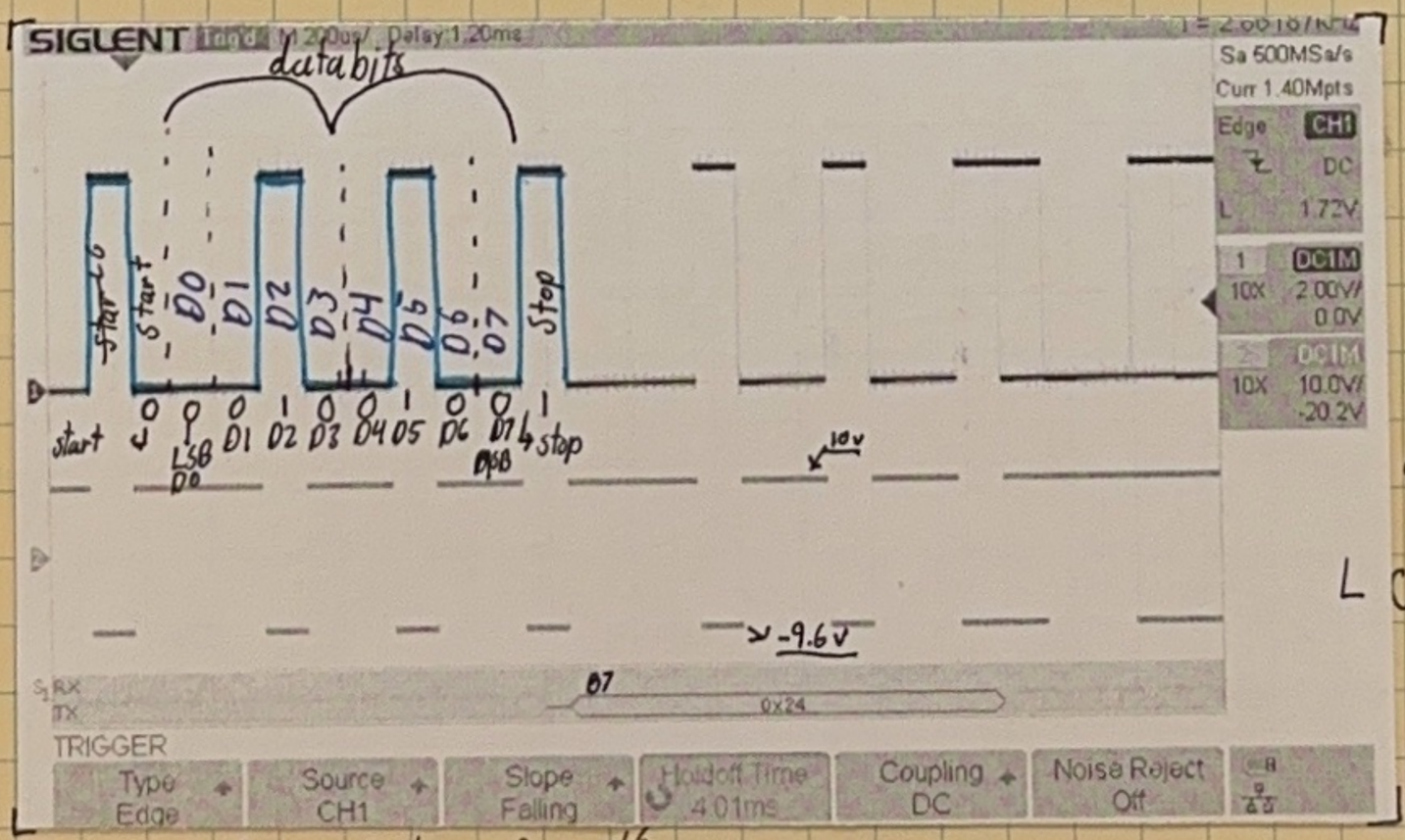
LG

Measure baud: 104.0039^{-1}
= 9615.024 baud

Error %
$\frac{9615.024 - 9600}{9600} \times 100\%$

error % = 0.156%

•Figure 4: Full schematic for UART & RS-232 Proof of life



LG

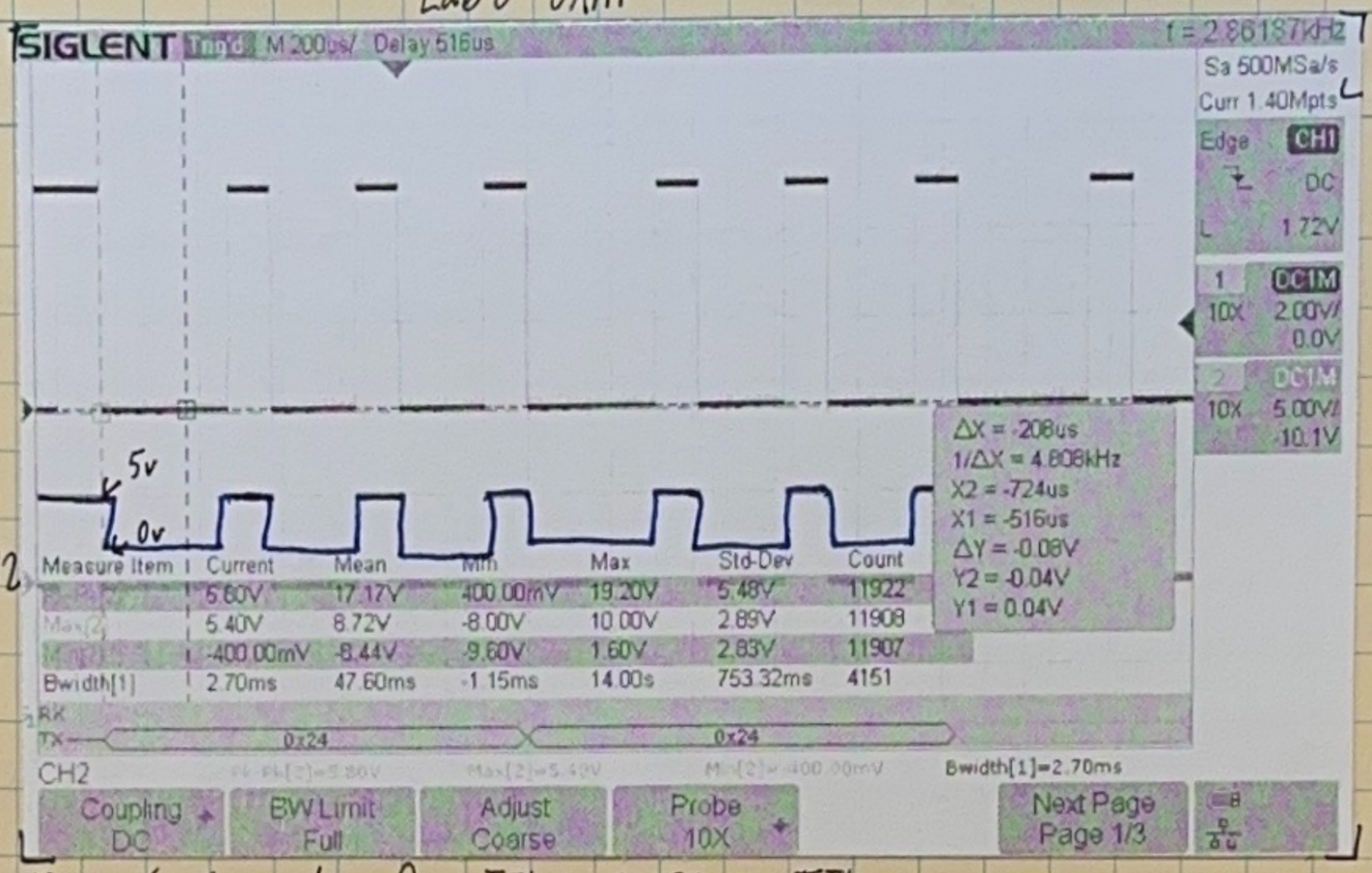
•Figure 5: Oscilascope Capture of ~~uart~~ UART Proof of life

-Testing the MAX232 to verify all the inputs and outputs work that the TTL can become RS-232 and the take RS-232 and bring it back to TTL. All ports work. Capacitors used were all the recommend for operation.

0 0 0 1 0 0 1 0 0 1
start LSB 4 4 2 MSB stop

10/12/2026
LRG

Lab 8 - UART



Channel 2 RS-232 to TTL

·Figure 6: Convertion from TTL to RS232 to TTL

```
Setup:
    ;           Bank 3
    BANKSEL ANSEL
    CLRF    ANSEL           ;set all pins to digital which doesn't do anything here.
    CLRF    ANSELH
    CLRF    BAUDCTL         ;clear register for Baud Rate Control Resgister
    ;           Bank 1
    BANKSEL TXSTA
    MOVLW   0b00100100      ;turn transmitter on, and asynchronus on with high speed,
    MOVWF   TXSTA           ; and 8 bit data
    MOVLW   25
    MOVWF   SPBRG
    BSF     TRISC,6         ;set as asynchronous serial output
    BSF     TRISC,7         ;set as asynchronous serial input
    ;           Bank 0
    BANKSEL RCSTA
    MOVLW   0b10000000      ;enable the serial port
    MOVWF   RCSTA           ;configures RX/DT and TX/CK pins as serial port pins
Main:
    BTFSS   PIR1,4          ;check to see if the transmit register is empty
    GOTO    $-1             ;if not keep waiting
    MakeItRain:
    MOVLW   0x24            ;send the "$"
    MOVWF   TXREG           ;write the transmission
    GOTO    Main            ;Re-run the program
    End
```

·Figure 7: Code and SFR's to send UART

·Shown in captures is the 8 data bits, 0 parity, 1 stop bit, 1 start bit and that equals 10 bits the data is sent LSB first.

D0=0, D1=0, D2=1, D3=0, D4=0, D5=1, D6=0, D7=0

·Also noted is the MAX232 inverts TTL to RS232 seen in Capture figure 5 on pg 130. More test's were done with diffenent hex values transmited.

TJR
10-12-26

10/12/2026
LRG

Lab 8 - UART

Part 2 - UART RX and Byte Echo:

Goal: Receive one byte from the host and immediately transmit the same byte back.

- Receive service time: The PIC16F883 receiver buffers two unread characters. Calculate the available service time continuous back to back traffic
- Using measured byte time from Part 1: 1.0417ms

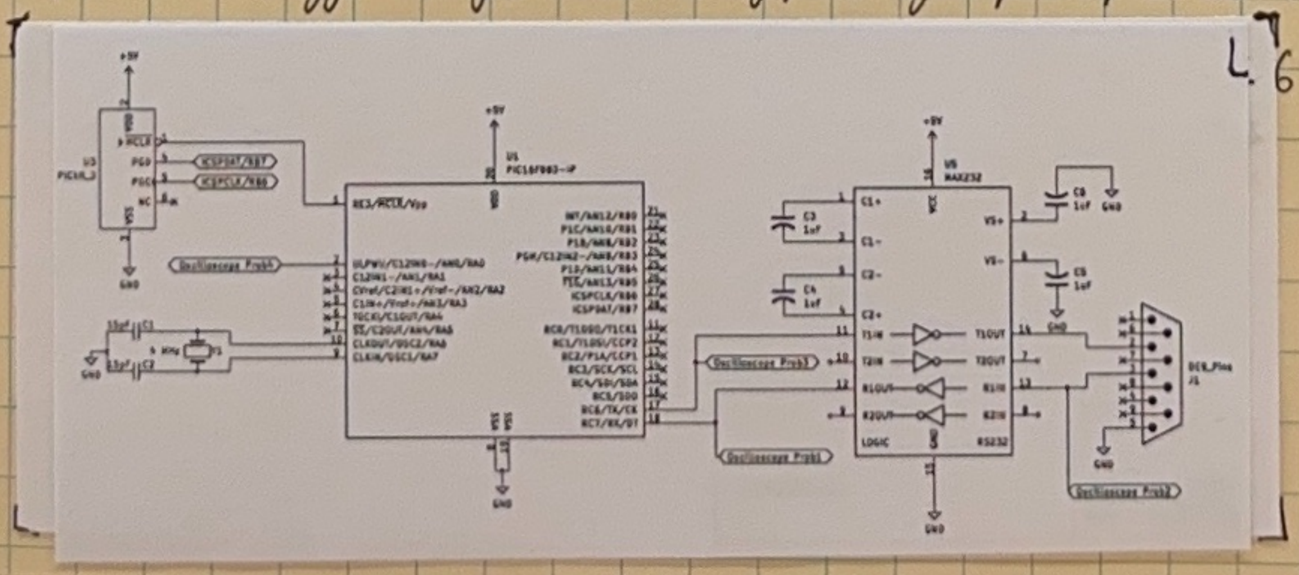
$$T_{\text{two byte window}} = (2(T_{\text{byte}}) = 2.083\text{ms}$$

- Two bytimes is the approximate window after the first unread byte is received. Service Rx at least once per character time, preferably more often.

- Required RAM, setup, and Program structure. See RCET3375/Lab../Lab8-UART.md and code figures in this section.
- The short pulse at the beginning of ServiceRx marks each receive-service check. Measure the time between consecutive pulse edges while the host sends back to back bytes. Compare the longest observed interva with your measured byte time and the two-byte receive window above.

- **Before Lab:**
  - RCSTA.CREN, PIR1.RCIF, RCREG, and the TX registers and flags used for echo.
  - receive FIFO/service-time calculation using your measured byte time.
  - diagnostic output pin selection and scope connection.
  - RX, echo-processing, and TX flowchart and source.

- **In the Lab:**
  1) Send a single byte from the host and verify that the PIC echoes it unchanged.
  2) Probe PIC RC7/RX and RC6/TX on separate oscilloscope channels. Trigger on the RX start bit and capture both the received and echoed frames in a single LG single acquisition.
  3) Compare the two frames and measure their bit times and the delay between the RX & TX start bit.
  4) Send several ASCII and binary byte values. Confirm that the host receives exactly the values it sent
  5) Probe the diagnostic output LG output while sending back to back bytes. Measure the longest interval between consecutive ServiceRx pulses and compare it with measured byte time and two byte receive window.
  6) Enable serial decoding for both oscilloscope channels at 9600 baud, 8N1. Compare the decoded RX and TX bytes.
  7) Experiment with hexedecimal, ASCII, and binary count displays, then try start bit and data byte trigger settings.
  8) Configure the oscilloscope to capture and decode RX and TX at both the TTL and RS-232 interfaces. Record the probe locations, trigger settings, decoder settings, and signal polarity for each interface.

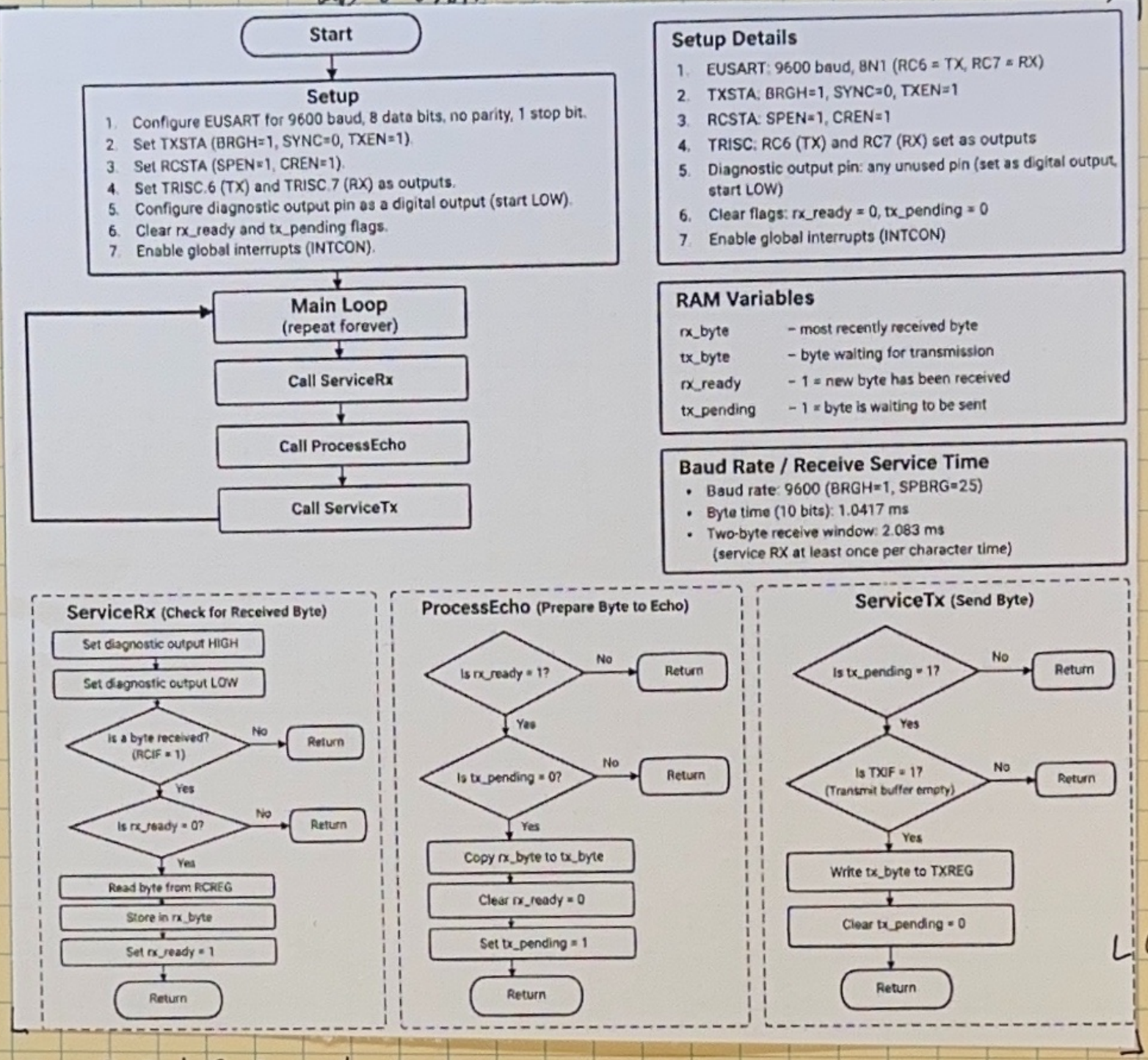


L.6

Figure 8: P2 schematic

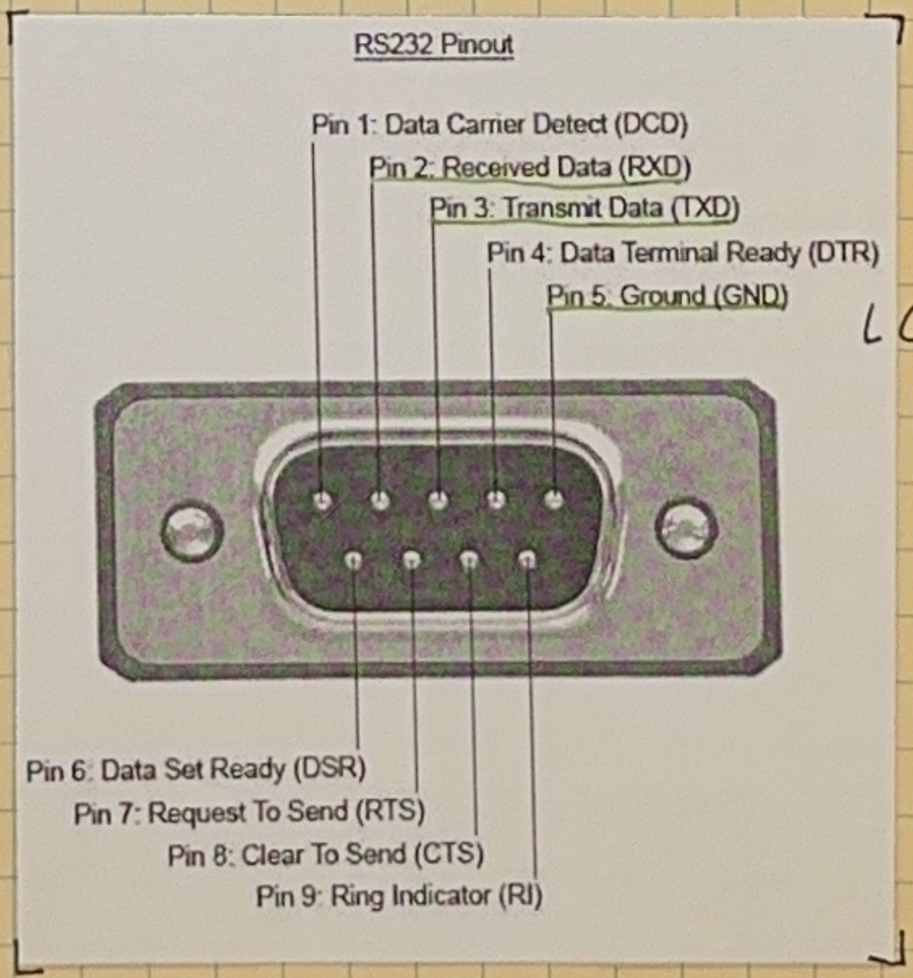
10/12/2026
LRG

Lab 8-UART



LG

•Figure 9: Part 2 flowchart of MPLAB operation code / Setup.



LG

•Pin 2, 3, 5 and 6 are the only pins used in this section.

•Figure 10: RS232 adaptor pinout

LG

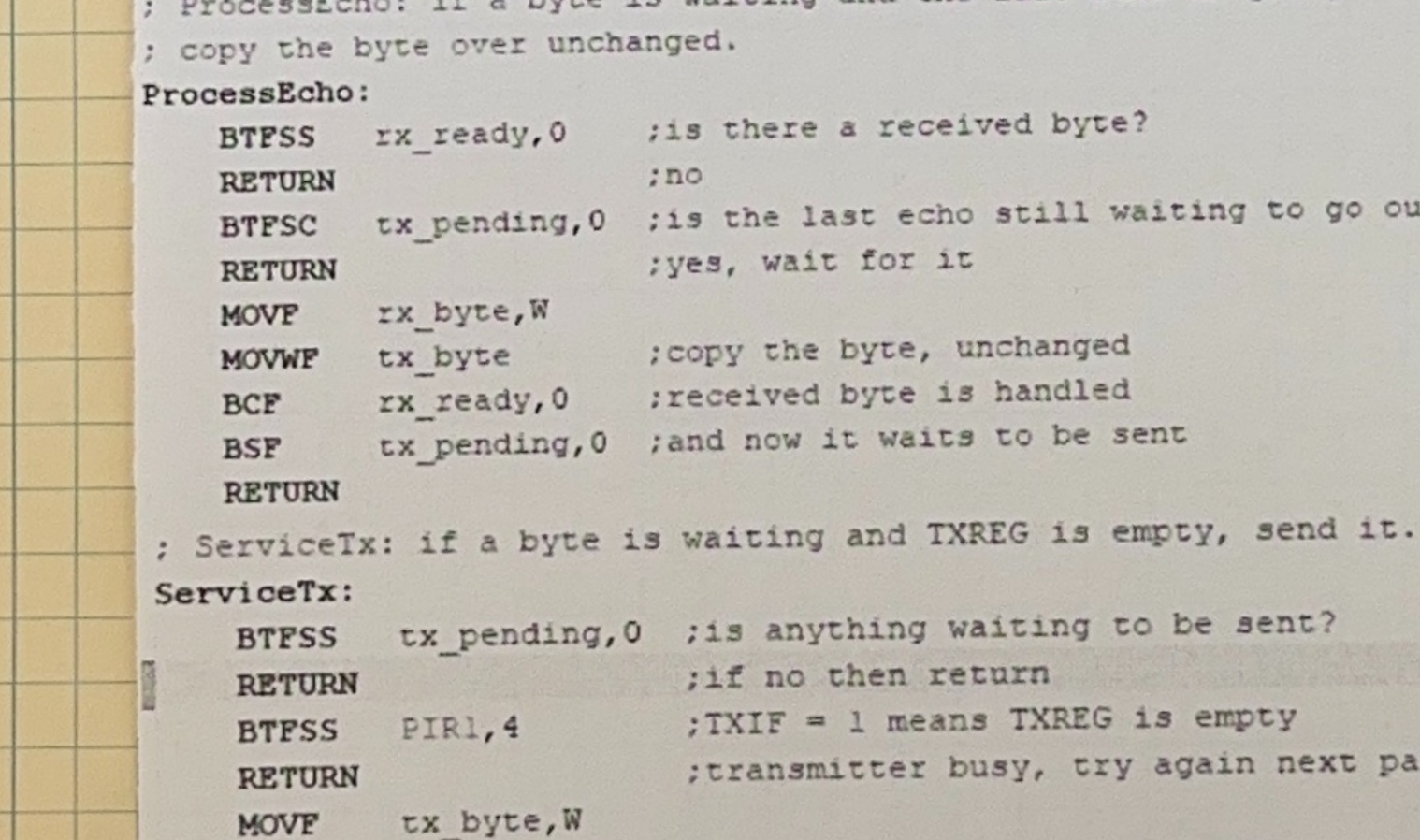
10/13/2026
LRG

Lab 8 - UART

10/13/2026
LRG

```
; ServiceRx: the pulse on RA0 marks every receive check. Then, only
; if the last byte was handled and a new one arrived, read it.
ServiceRx:
    BANKSEL PORTA
    BSF     PORTA,0         ;diagnostic pulse HIGH
    BCF     PORTA,0         ;and LOW on the very next instruction
    BTFSC   rx_ready,0      ;is the last byte still waiting to be handled?
    RETURN                  ;yes, don't overwrite it
    BTFSS   PIR1,5          ;RCIF = 1 means a byte has arrived
    RETURN                  ;nothing arrived, nothing to do
    MOVF    RCREG,W         ;read the byte, this also clears RCIF
    MOVWF   rx_byte
    BSF     rx_ready,0      ;tell ProcessEcho there is a byte
    BTFSS   RCSTA,1         ;OERR = 1 means the receiver overran
    RETURN                  ;no overrun, done
    BCF     RCSTA,4         ;overrun: turn CREN off
    BSF     RCSTA,4         ;and back on, which clears OERR
    RETURN
; ProcessEcho: if a byte is waiting and the last echo is gone,
; copy the byte over unchanged.
ProcessEcho:
    BTFSS   rx_ready,0      ;is there a received byte?
    RETURN                  ;no
    BTFSC   tx_pending,0    ;is the last echo still waiting to go out?
    RETURN                  ;yes, wait for it
    MOVF    rx_byte,W
    MOVWF   tx_byte         ;copy the byte, unchanged
    BCF     rx_ready,0      ;received byte is handled
    BSF     tx_pending,0    ;and now it waits to be sent
    RETURN
; ServiceTx: if a byte is waiting and TXREG is empty, send it.
ServiceTx:
    BTFSS   tx_pending,0    ;is anything waiting to be sent?
    RETURN                  ;if no then return
    BTFSS   PIR1,4          ;TXIF = 1 means TXREG is empty
    RETURN                  ;transmitter busy, try again next pass
    MOVF    tx_byte,W
    MOVWF   TXREG           ;writing here starts the transmission
    BCF     tx_pending,0    ;it is on its way
    RETURN
    End
```

•Figure 11: The RX and TX service code and Process the echo.



•Measured Tbyte= ~~1.083ms~~ 2.048ms
•Predicted Tbyte 2.084

•Figure 12: Oscilloscope capture of RS232 data to TTL RX which is echo on Tx back

10/13/2026
LRG

Lab 8 - UART

```
Setup:
    ;       Bank 3
    BANKSEL ANSEL
    CLRF    ANSEL           ;set all pins digital, RA0 needs this as an output
    CLRF    ANSELH
    CLRF    BAUDCTL         ;set the 8-bit baud rate generator
    ;       Bank 1
    BANKSEL TXSTA
    MOVLW   0b00100100      ;transmitter on, asynchronous, high speed,
    MOVWF   TXSTA           ;and 8 bit data
    MOVLW   25
    MOVWF   SPBRG           ;9600 baud divisor, 9615 actual at 4 MHz
    BCF     TRISC,6         ;bit set as the data sheet says for the TX pin
    BSF     TRISC,7         ;bit set as the data sheet says for the RX pin
    BCF     TRISA,0         ;RA0 is an output, the diagnostic pin
    ;       Bank 0
    BANKSEL RCSTA
    MOVLW   0b10010000      ;SPEN = 1 enables the serial port,
    MOVWF   RCSTA           ;CREN = 1 turns on continuous receive
    CLRF    PORTA           ;diagnostic pin starts LOW
    CLRF    rx_ready        ;no byte received yet
    CLRF    tx_pending      ;nothing waiting to send
Main:
    CALL    ServiceRx       ;check the receiver
    CALL    ProcessEcho     ;hand a received byte over to the send side
    CALL    ServiceTx       ;send it if the transmitter is free
    GOTO    Main            ;repeat forever
```

LG

•Figure 13: SFR's and Main code for operation the call to code see pg 134.

• Use our serial send C# Program that using a ASCII ~~sybte~~ symble like ($ LG (24)) which should send a hex valve of 0x24. The program on the pic is to echo the captured byte received on the Rx and echo it back on the Tx. We used a RS232 adapter and Max 232 to convert to TTL. The adapter had noise which is ok. The signal came through clear. PORTA was a diagnostic bit that continuously runs until the echo occurs. Once Rx stop is set high then it takes the Received byte and (TX) transmits it. the longest gap was 41µs. The Tested with othe ASCII values like U=0x55.

TJR
10-13-26

-The Trigger setting were set to serial and in the Protocol is UART making sure the correct Signal Rx and Tx is setup and trigger setting is either set to Rx or Tx here set to Rx start condition.

•Part 3 - Structure Packets and ADC Data

•Goal: Replace byte echo with a nonblocking packet parser. The host will request the PIC's device ID or ful 10bit ADC reading from the potentiometer on AN0.

-Packet format: Each Packet contains

$	COMMAND	LENGTH	DATA....
24		(hex start bit)	

•$ is the start byte (0x24)

•LENGTH is the number of data bytes, not including first 3 packet bytes

•Numeric fields are binary bytes, not printable ASCII digits. •Multi-byte values are sent high byte first.

•The receiver processes one byte per call and remembers its parser state between bytes. Requests may arrive with pauses between bytes.

•The host sends one request and waits for its response before sending another.

•Unknow commands, incorrect lengths, and channels other than AN0 are ignored. The host should handle a missing response with a timeout.

10/13/2026
LRG

10/14/2026
LRG

Lab 8-UART

-At 9600 baud rate, 8N1, a three byte ID request takes:

•$T_{packet} = (3 + LENGTH)\ T_{byte}$

$3 \cdot 1.0417ms = \underline{3.125ms}$

-The time needed to receive a whole packet differs from the time available to service each received byte.

•Define Part 3 packets:

-The field and hexadecimal forms below describe the same bytes. HH and LL are the two bytes of the right Justified ADC result.

```
Request:   $ I 0                    24 49 00
Response:  $ I 2 0x33 0x75          24 49 02 33 75

Request:   $ A 1 0x00               24 41 01 00
Response:  $ A 3 0x00 HH LL         24 41 03 00 HH LL

Example for ADC = 0x02AB:           24 41 03 00 02 AB

Framing:   $ E 2 0x01 0x00          24 45 02 01 00
Overrun:   $ E 2 0x02 0x00          24 45 02 02 00
```

} •Device ID (fixed at 0x3375)

} •AN0 ADC reading (channel 0x00)

} •UART receiver error

•Figure 14: The ID; for Device, AN0 ADC, and UART receive errors

-A receive error discards the current partial requests and resets the parser to wait for $. The host can resend the request. Both error packets use a zero detail byte.

•Non-blocking operation and indexed buffers-

-Nonblocking: Each service routine checks whether work can be done now and returns promptly rather than waiting for another RX byte, an ADC conversion, or an available TX register. The diagnostic pulse at the start of each main loop pass marks successive opportunities to service RX. Measure the longest interval between pulses while packets are being received, ADC conversions are running, and replies are transmitted. Compare the interval with the receive-service timing budget from Part 2; a long gap can reveal blocking code.

-Buffer, index, and pointer: tx_data 0-5 is a six byte buffer in consecutive RAM locations. tx_count in the index of the next byte to transmit. The PIC16F883's FSR (File Select Register) can hold the address of tx_data[tx_count], acting as a pointer. Accessing INDF then reads or writes the RAM byte at that address; this is indirect addressing. For example, when tx_count = 2, pointing FSR to tx_data[2] makes an INDF read access the third packet byte. See the PIC16F883 datasheet, Section 2.4, "Indirect Addressing, INDF and FSR Registers" (page 38).

•Required RAM, Setup, and Program structure is all within code and to see requirements go to timor-rosstime github/RCET3375/Lab Assignments/Lab 8-UART.md.

Lab 8- UART

•Graphical host application:

-Develop a graphical host application to comunicate with the PIC. The interface must provide!

•Connection controls: select the serial port, connect/disconnect at 9600 baud (8N1), and show connection status.

•Packet controls: send the defined Device ID and AN0 requests, and enter/send a custom sequence of hexadecimal bytes for protocol testing. Transmit actual byte values, not ASCII representation of hex digits.

•Decode results: display the device ID and the latest AN0 ADC reading as a 10-bit integer (0-1023). Identify framing/overrun error replies, invalid or incomplete responses, and request timeouts.

•Transaction history: show sent and received packets in a scrollable list of table with TX/RX direction, hexadecimal bytes, and a decoded description or status. Keep multiple transactions visible for inspection.

-Use the packet LENGTH field to assemble complete replies even when received bytes arrive separately. Send one request at one time, wait for its response or a timeout, and keep ~~multiple transactions visible~~ for inspection. the GUI responsive while waiting. The application should be reusable in part 4 when temperature conversion and servo commands are added.

•Before Lab:

•packet definitions, packet time calculations, and parser flowchart.

•RAM map and a method for indexing the six byte TX buffer, including an example of how FSR/INDF can access an index byte

•working LAB 7 AN0 ADC code and full 10-bit result handling.

•PIC source and a graphical host application design showing the connection controls, packet commands, decode results, and transaction history.

•In the Lab:

1) Build and run the graphical host application. Connect to the PIC and use its controls to send, receive, and inspect packets.

2) Request the device ID and verify 0X3375 is returned and decoded correctly.

3) Request AN0 repeatedly while turning the potentiometer. Verify the displayed ADC reading approaches 0 and 1023 near the ends of travel.

4) Compare several returned ADC values with the AN0 voltage measured using the lab 7 method.

5) Use the custom packet control to send an unknown command, an incorrect length, and a request for channel 0x01. Verify that the PIC ignores them, the host reports a timeout, and the next invalid requests succeeds.

6) Measure the longest interval between receive-service diagnostic pulses while receiving requests, preforming ADC conversions, and transmitting replies. Compare it with the byte time and two-byte receive window from part 2.

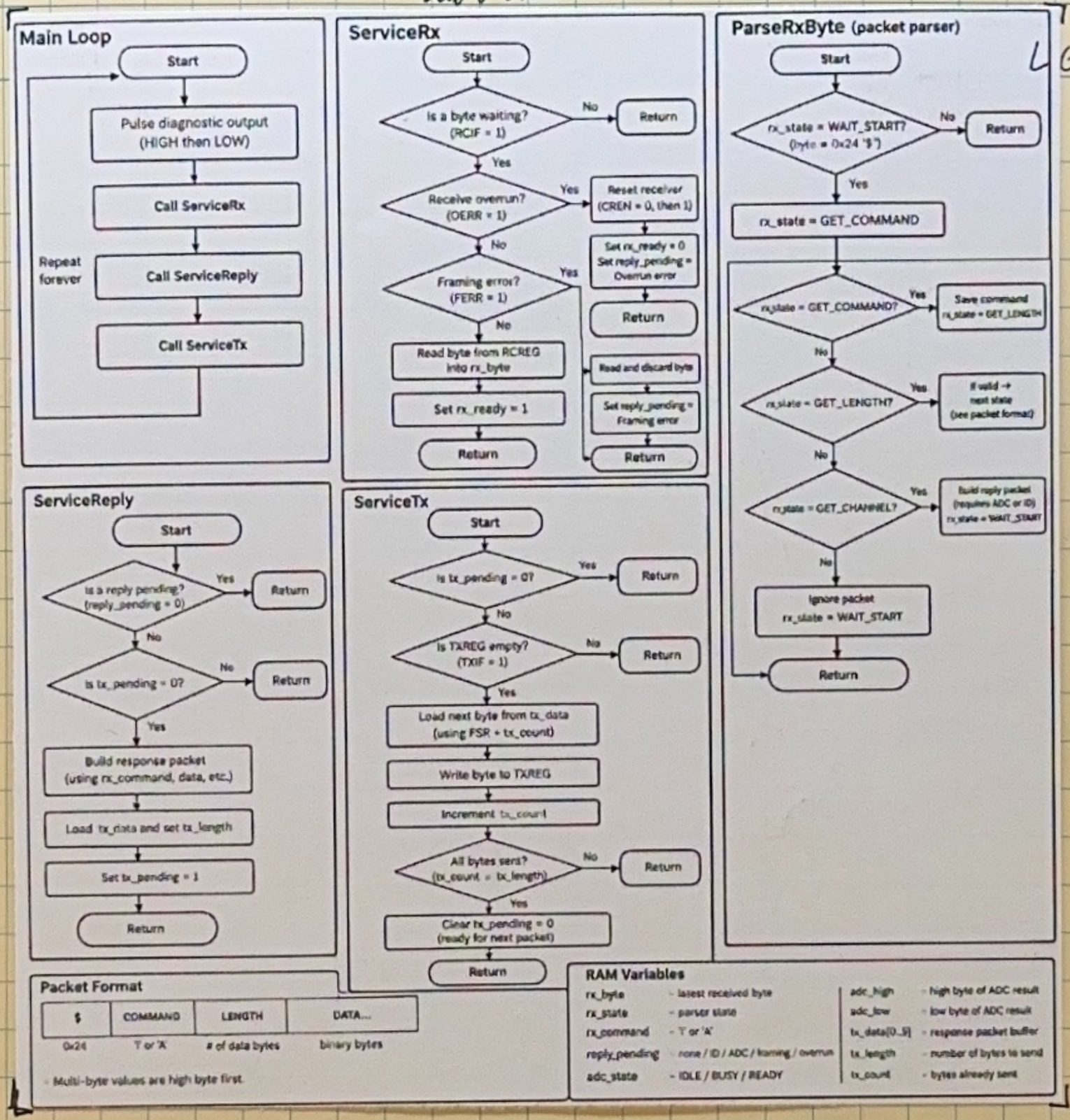
7) Temporarily delay RX servicing while the host sends at least three bytes back to back. Verify the overrun packet (24 45 02 02 00), restore normal servicing, and confirm recovery with a valid request.

8) Cause a framing error by using a baud mismatch (4800 baud). Check for the error framing packet (24 45 02 01 00), the restore 9600 baud and confirm recovery.

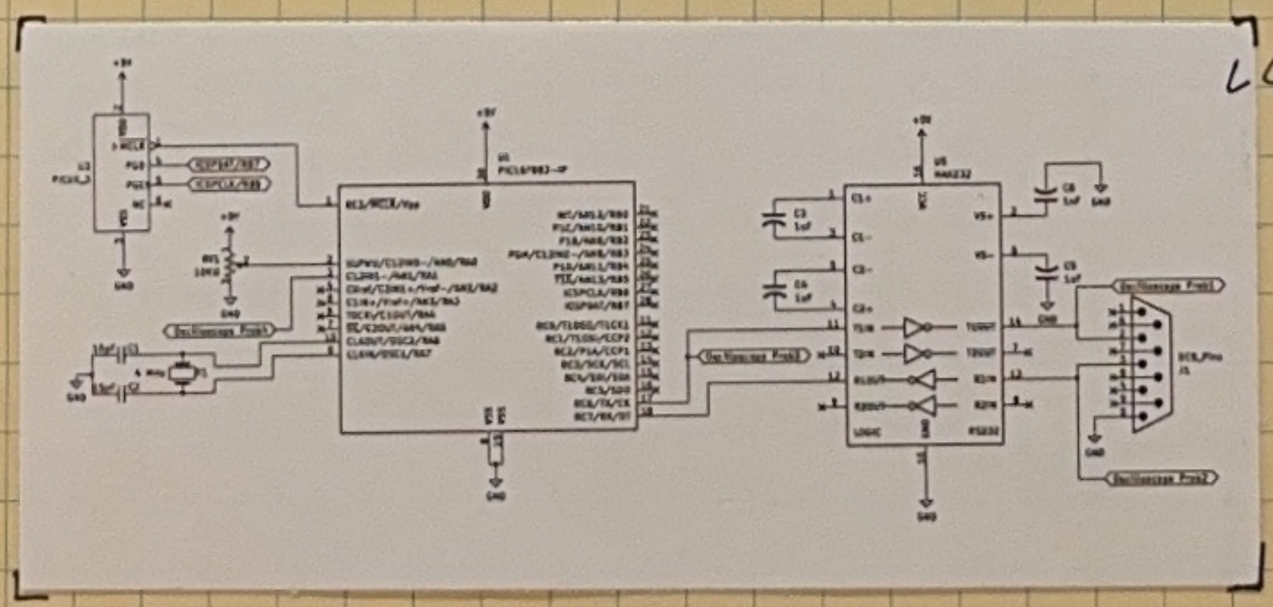
9) Capture a screenshot of the GUI showing several packet transcations, including successful ID and ADC reponses and a timeout error or error indication.

Lab 8-UART

10/15/2026
LRG



•Figure15: Flow chart for mapped program structure of Part3 Structure Packets & ADC Data



•Figure16: Full schematic for Structure Packets & ADC Data

-The MPLab assembly code is designed so the PIC over a serial port at 9600 baud, 8N1. It can ask the PIC for its Device ID or for the AN0 (potentiometer) ADC reading, and it shows every packet sent and received. The flow of code works as follows (with C# interface), Connect opens the picked port, Next a request button (either Send ID, of check ADC) will call the Send bytes, which writes real byte values to the port and starts a 0.5 second timeout. The Status timer tick runs every 20ms and checks the if any new arrived bytes are in the rx buffer. Checks for the whole packet or if it takes to long the a timeout error shows. Only one request is sent at a time. The error packet of overrun has to erable be enabled in the MPLab code all it is, is a comment ";" needs to be removed for this to have an overrun.

10/15/2026
LRG

```
Setup:
    ;       Bank 3
    BANKSEL ANSEL
    MOVLW   0b00000001    ;only AN0 analog, RA1 digital (diagnostic)
    MOVWF   ANSEL
    CLRF    ANSELH
    CLRF    BAUDCTL       ;BRG16 = 0 (8-bit baud generator)
    ;       Bank 1
    BANKSEL TXSTA
    MOVLW   0b00100100    ;TXEN=1, BRGH=1, 8-bit, async
    MOVWF   TXSTA
    MOVLW   25            ;9600 baud @ 4 MHz (actual ? 9615)
    MOVWF   SPBRG
    BSF     TRISC,6       ;TX pin
    BSF     TRISC,7       ;RX pin
    BSF     TRISA,0       ;RA0 AN0 input
    BCF     TRISA,1       ;RA1 diagnostic output
    MOVLW   0b10000000    ;ADFM=1 (right-justified), VCFG=00 (VDD/VSS)
    MOVWF   ADCON1
    ;       Bank 0
    BANKSEL RCSTA
    MOVLW   0b10010000    ;enable the serial port and continuous receive
    MOVWF   RCSTA
    MOVLW   0b01000001    ;ADC clock = Fosc/8, channel = AN0, ADC turned on
    MOVWF   ADCON0
    CLRF    PORTA         ;start with the diagnostic pin low
    CLRF    rx_state      ;parser begins looking for a '$'
    CLRF    rx_command
    CLRF    reply_pending ;no answer waiting to be sent
    CLRF    adc_state     ;ADC is idle
    CLRF    adc_high
    CLRF    adc_low
    CLRF    tx_length     ;nothing in the transmit buffer yet
    CLRF    tx_count
```

LG

•Figure 17: SFR's and Rewired set up code for Part 3.

```
ParseRxByte:
    WAIT_START:
        if byte is '$', advance rx_state to GET_COMMAND
        return
    GET_COMMAND:
        if byte is 'I' or 'A':
            save command; advance rx_state to GET_LENGTH
        otherwise:
            reset rx_state to WAIT_START
        return
    GET_LENGTH:
        if command is 'I' and received length byte is 0:
            reset rx_state to WAIT_START
            if no reply is pending:
                reply_pending = ID
        else if command is 'A' and received length byte is 1:
            advance rx_state to GET_CHANNEL
        otherwise:
            reset rx_state to WAIT_START
        return
    GET_CHANNEL:
        reset rx_state to WAIT_START
        if received channel byte is 0 (AN0) and no reply is pending:
            reply_pending = ADC
            set adc_state to IDLE
        return
```

LG

•Figure 18: The Parse Rx Byte Program Structure for Part 3

Lab 8 - UART

10/15/2026
LRG

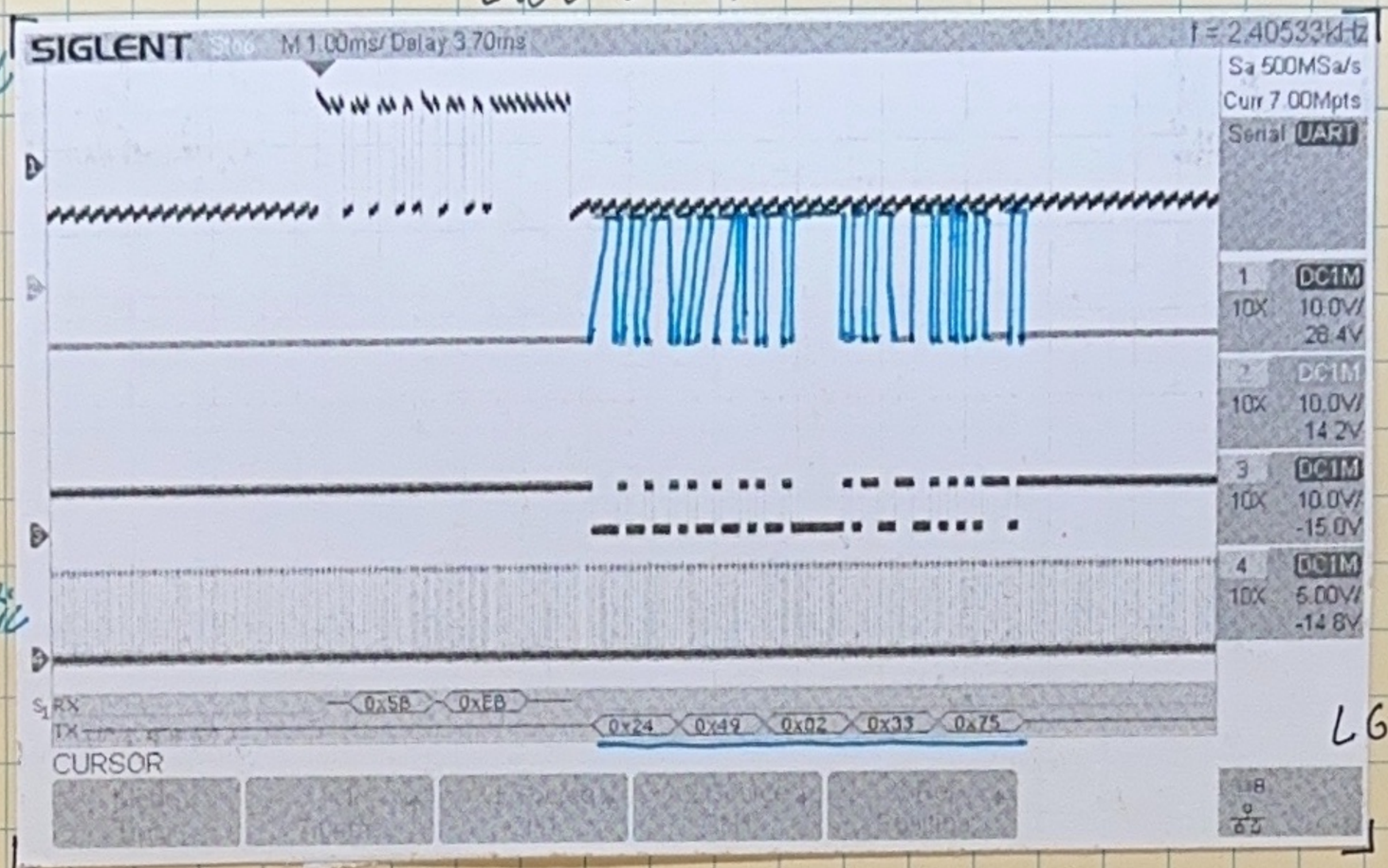
From PIC

Rx
RS-232

Tx
RS-232

Tx
TTL

Diagnostic



LG

• Figure 19: Oscilloscope Capture of Packet Request and Response for Device ID

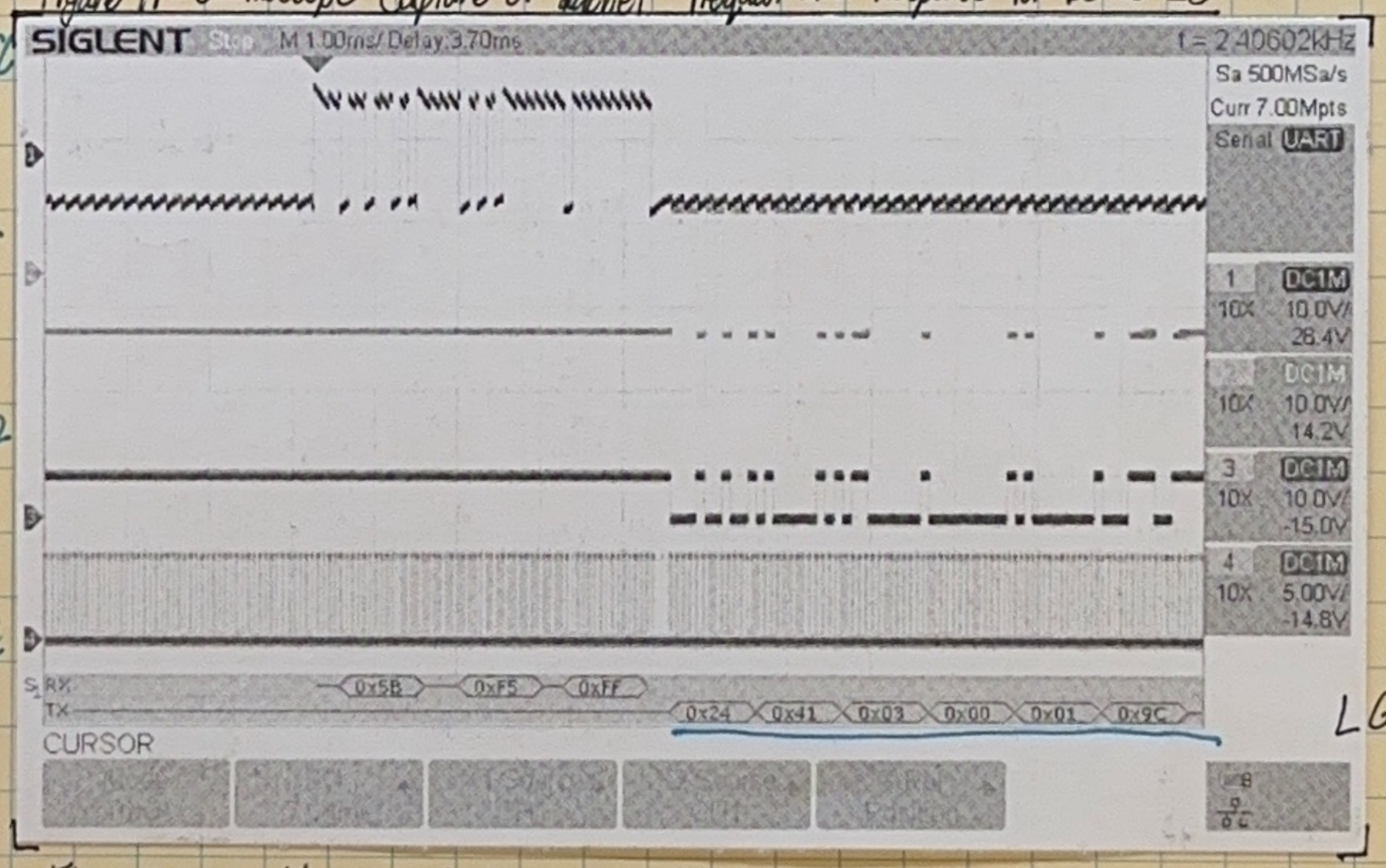
From PIC

Rx
RS-232

Tx
RS-232

Tx
TTL

Diagnostic



LG

• Figure 20: Oscilloscope Capture of Packet Request and Response for AN0 ADC reading

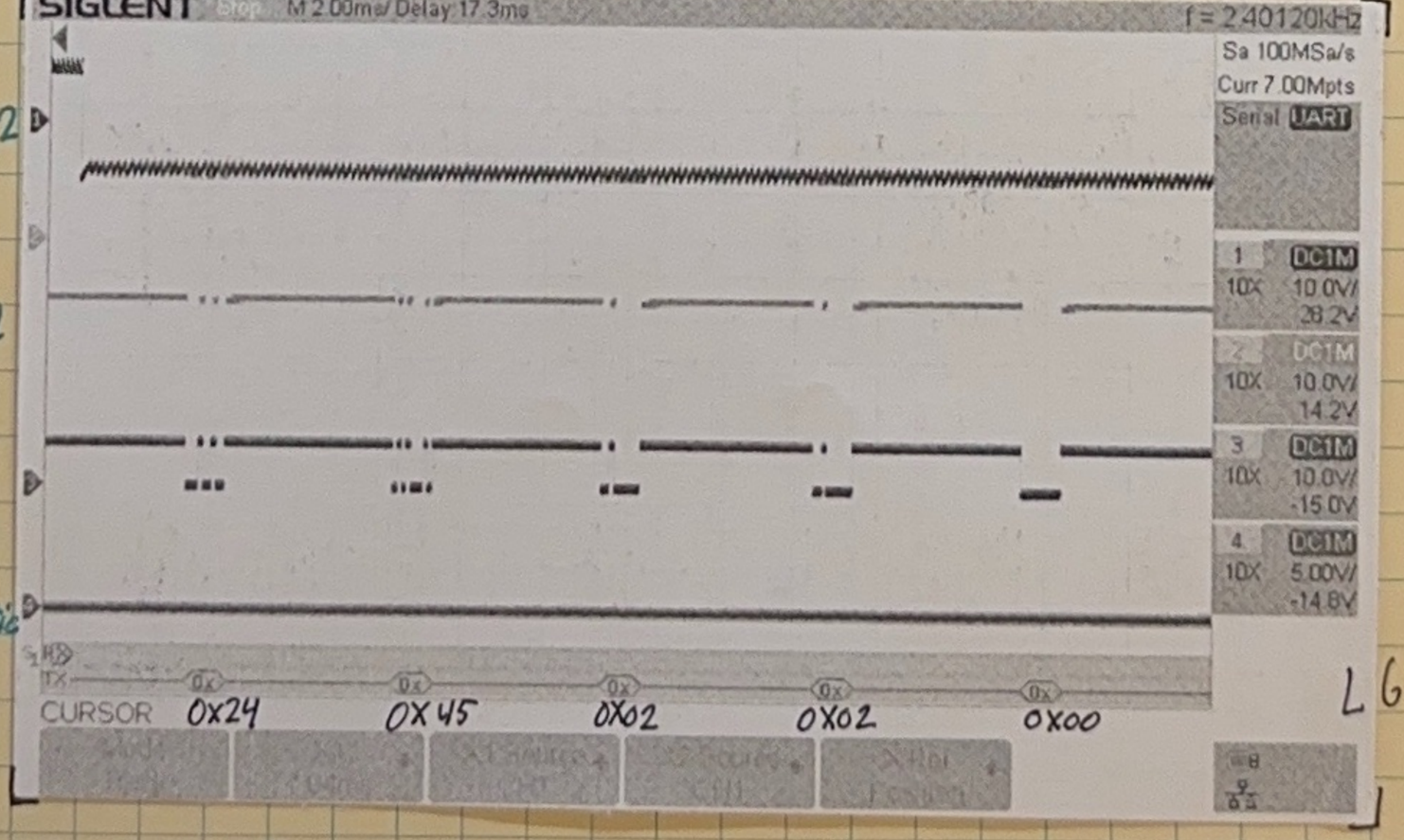
Rx
RS-232

LG

Tx
RS-232

Tx
TTL

Diagnostic



LG

• Figure 21: Oscilloscope Capture of Overrun error Packet

• The diagnostic bit in portA helps verify how often Main runs.

10/15/2026
LRG

Lab 8 - UART

```
TX  24 41 01 00
RX  24 41 03 00 02 14  ADC = 532
TX  24 41 01 00
RX  24 41 03 00 00 00  ADC = 0
TX  24 41 01 00
RX  24 41 03 00 03 FE  ADC = 1022
TX  24 49 00
RX  24 49 02 33 75  Device ID = 0x3375
TX  00  Sent at 4800 to force FRAMING
TIMEOUT: no reply
```

6 • ADC conversion check

Predicted	measured
• 0v = 0	• 0.001v = 0
• 2.5v = 512	• 2.504v = 510
• 5v = 1023	• 4.995 = 1021

• Figure 22: GUI capture of ADC measurement / Device ID / Framing error

• Here is the GitHub link to the C# interface code and MPLab code =
- https://github/groeluke/Lab_8_UART.git (sections Part 3 and commit on Oct. 15 12:30 for Serial data).

- The packets were hard coded in MPLab so when it was Servicing Rx it allowed for it to check at the PIC if anything was received and the check that information so it knows how to respond. Once all operations were checked and verified all cod was push and git commits were done for a clean working tree in git bash.

- The Overun and Framing errors are easy shown on the oscilloscope. Also verified that the ADC is operating correctly.

• Part 4 - LM34, UART, & Servo Integration

• Goal: Replace the potentiometer with an LM34 tempsensor and integrate UART RX/TX, ADC acq "acquisition, and one CCP-timed servo. The UART packet structure remains unchanged. The PIC returns raw ADC data. Temperature conversion and calibration are preformed on the host.

TJR
10-15-26

• LM34 conversion:
- The LM34 nominal output scale is : 10mV / °F
- Using VDD/VSS as ADC references: $V_{sensor} = ADC\left(\frac{V_{REF}}{1024}\right)$
- Then: $T[°F] = \frac{V_{sensor}}{0.01v/°F}$

Worked example =
- What: convert one returned ADC result to temperature.
- Why: the PIC transmits raw measurement data; the host assigns the engineering units.

$$V_{sensor} = 150\left(\frac{5}{1024}\right) = 0.732V$$
$$T = \frac{0.732V}{0.010v/°F} = 73.2°F$$

- Use measured reference voltage and calibration results.

• Servo command packet: the host converts the requested pulse width to an absolute CCP1 compare value. At 4 MHz, using Lab07's 1µs Timer1 tick and reload value of 0xB1E0:

CCP1 match = 0xB1E0 + PW_US
45536 + 1500
= 47036 = 0xB7BC

10/15/2026
LRG

-The s command contains two payload bytes, the high byte and then the low byte of the absolute match:

```
Command:                $ S 2 MATCH_H MATCH_L
1500 us command:         24 53 02 B7 BC      (no reply)
Below minimum:          24 53 02 B3 D3
Above maximum:          24 53 02 BB A5
Invalid servo reply:    24 45 02 03 01
```

•Figure 23: Command for servo ranges.

Servo HIGH time	Allowed CCP match
500 µs	0xB3D4
1500 µs	0xB7BC
2500 µs	0xBBA4

•Table 2: For Servo time to given hex value

-The PIC accepts the full 500-2500µs range, including both endpoints. Reject absolute compare values outside 0xB3D4 - 0xBBA4; leave the last accepted setting unchanged and return 25 45 02 03 01. Valid s commands commands have no response. Preserve the Part 3 ID, ADC, and receive error packet.

•Either Firmware integration: Extend the working code from Part3 and Lab07. Combine their existing subroutines and interrupts to implement the new behavior.

-Packet parser: Add a 's' with length 2, followed by the high and low compare bytes. Collect both bytes before validating the 16-bit value. The I and A commands continue working as in Part3.

-New servo target: Reuse Lab07's timer1/CCP1 20ms frame, 1µs resolution, RC2 servo output, and it initial 1500µs setting. Instead of the Lab07 ADC lookup/calculation for the servo, take the next compare value from a valid UART s packet.

-Next frame update: Store the validated compare bytes together in RAM, protecting the two byte update from the an interrupt. the Timer1 frame start interrupt transfers that saved value into CCPR1H:CCPR1L. Receiving a command must not write the hardware compare registers or disturb a pulse in progress.

-Foreground servicing: Retain Lab07's servo_busy behavior. Don't start UART/ADC foreground service work while the service work while the servo output is High; check the busy flag between service calls. Keep the Part3 receive-service diagnostic output on RA1.

-Existing functions: Preserve the Part3 ADC service, packet responses, and receive-error recovery.

•Extend the graphical host application: Build on the Part3 GUI and preserve its packet decoding, error reporting, and scrollbar transaction history. In Part4, the application much operate automatically without requiring the user to choose a serial port or press connect.

-Automatic PIC Identification: On Startup, enumerate available serial ports, open each candidate at 9600 baud (8N1), and send the device-ID request 24 49 00. A port is accepted only when the complete response is exactly 24 49 02 33 75. Close ports that don't return that ID. Then display the verified port (like (COM5)) and the connection status.

10/15/2026
LRG

Lab 8-UART

-Temperature: Once the PIC is identified, request AN0 (24 41 01 00) every 1 second. Convert the returned 10bit ADC reading into a temperature using the measured reference voltage and display the result in °F. Keep the display temperature current.

-Servo slider: Positions 0°-180° inclusive in 1° increments, with the slider initially set to 90°. Map 0° to 500µs, 90° to 1500µs, and 180° 2500µs, interpolating between these points. Display the degree setting. Convert the pulse width to 0xB1E0 + PW, then send high and low bytes using 24 53 02 MATCH_H MATCH_L.

-Communication loss: If the port is lost or communication fails, show COM ERR and place the temperature with ---°F. Close the failed connection and attempt PIC discovery every 1 second until a valid ID reply is received. On reconnection, show the newly verified port and resume temperature updates automatically, without assuming that the old port name is still valid.

•Before the Lab:

•schematic, loading calculation, and the LM34, MAX232, PIC, and servo information

•LM34 temperature and servo compare-value worked calculation and predicted measurments

•Working Part3 and Lab07 projects, a plan for integrating the S packet and next frame update, and updating the parser, interrupt flow charts.

•The GUI design for ID-based recovery, 1 second °F updates, and 0-180 slider, full packet logging, and automatic reconnection.

•In the Lab:

1) Combine Part3 with the Lab07 servo timing and add the S packet. Verify the original ID, ADC, and error response still work.

2) Build the GUI to the requirments above. Replace the pot with the LM34, measure the ADC reference and sensor voltage, and compare the GUI temp with an independent reference.

3) With the servo disconnected, command 500µs, 1500µs, and 2500µs measure each High pulse and the 20ms frame on the oscope. Verify the new command affects a subsequent frame without changing the pulse in progress.

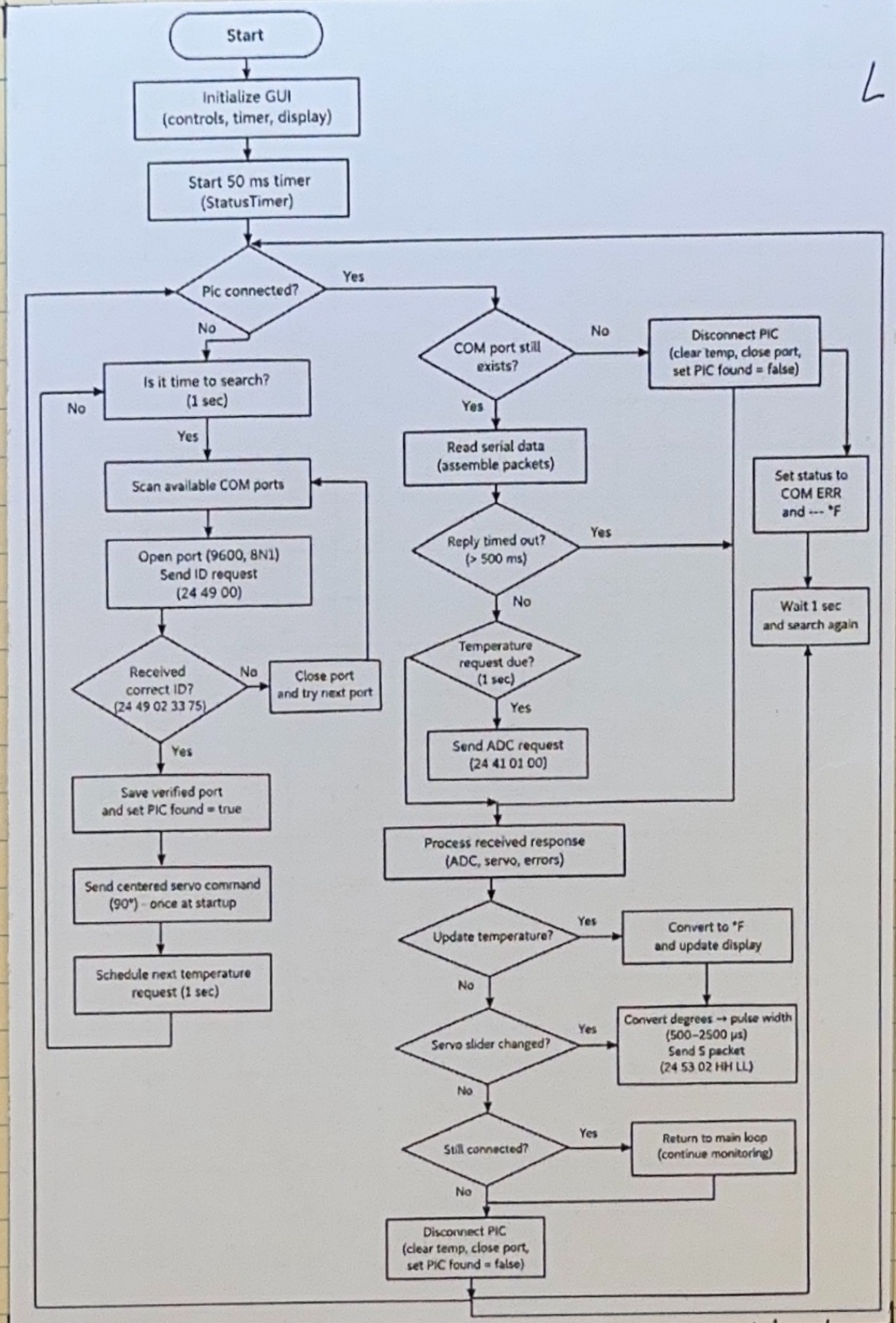
4) Send B3 D3 and BB A5, check for the expected error replay, and confirm that the last valid servo setting remains in effect.

5) Start the GUI with the PI adapted already connected. Confirm automatic ID verification, display port, 90° initial slider, one startup servo packet, 1second temp updates, and servo packets only when the slider changes.

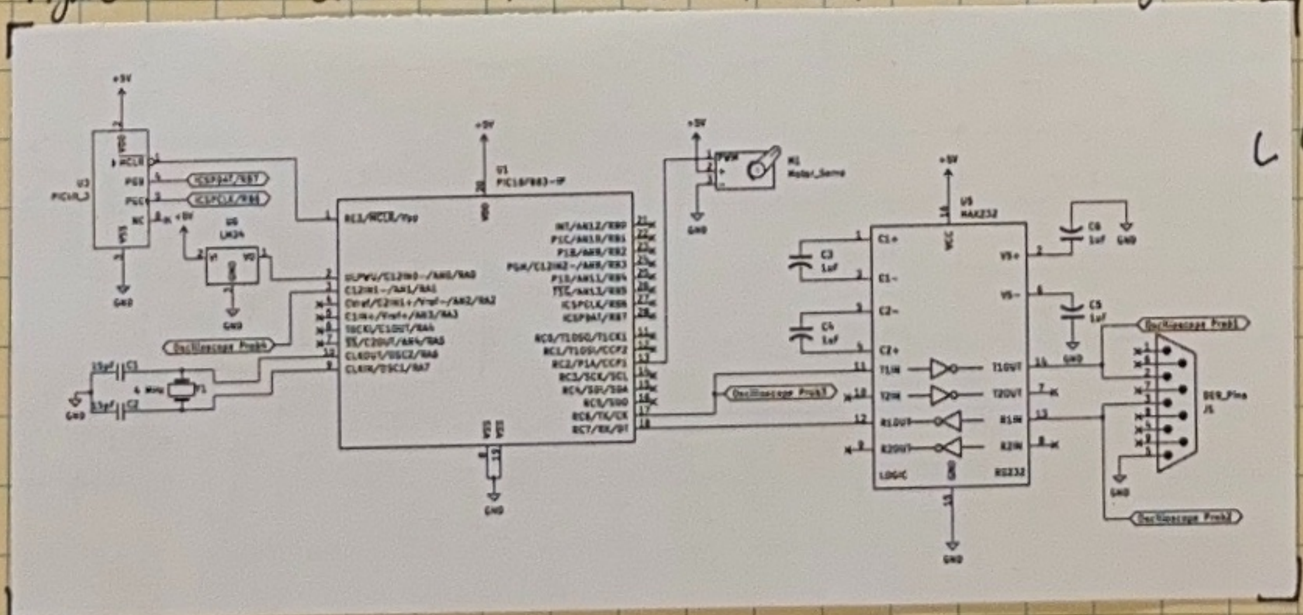
6) Unplug the USB-to-RS 232 adapter from the host PC while the GUI is running. Verify Com ERR, ---°F, 1-second reconnection attempts, and automatic recovery after reconnection

7) Attach at least one additional virtual serial port device. Repeat discovery with the PIC connected and disconnected; confirm that only an ID-verified PIC port receives ADC or servo commands.

8) Measure RC2 and the RA1 receive-service diagnostic output together at different pulse widths. Compare the longest RX service interval with your part2 timing calculation. Note any com errors, their conditions, repeatability, and what happens after a fresh valid request.



•Figure 24: Flow Chart for Part 4 LM34, UART, and Servo integration



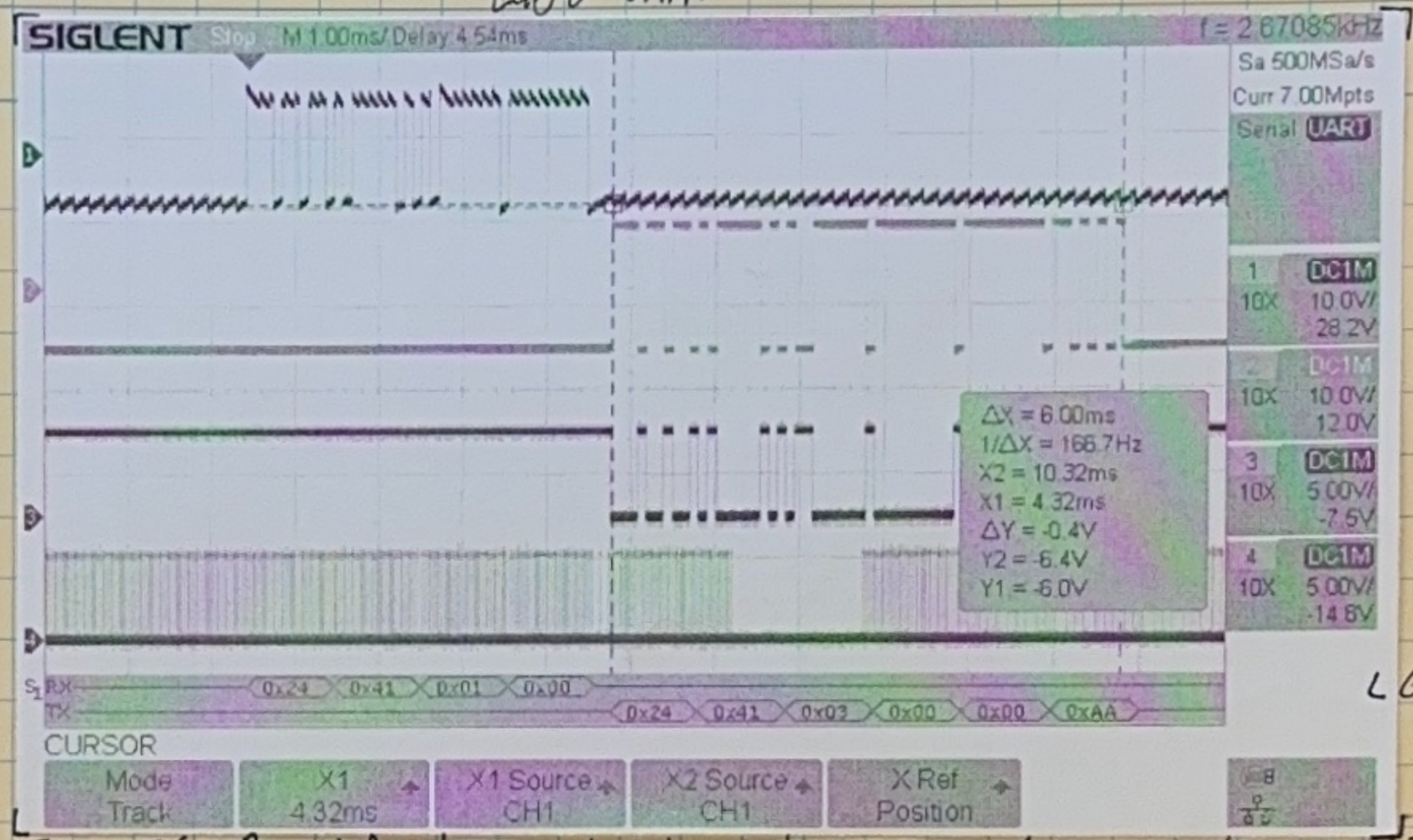
•Figure 25: Full Schematic for Part 4 LM34, UART, and Servo integration

		MIN	MAX	UNIT
Supply voltage		35	–0.2	V
Output voltage		6	–1	V
Output current			10	mA
Storage temperature, T_{stg}	TO-46 Package	–76	356	°F
	TO-92 Package	–76	300	
	SO-8 Package	–65	150	

•Table 3: LM34 current, voltage, and temp spec's

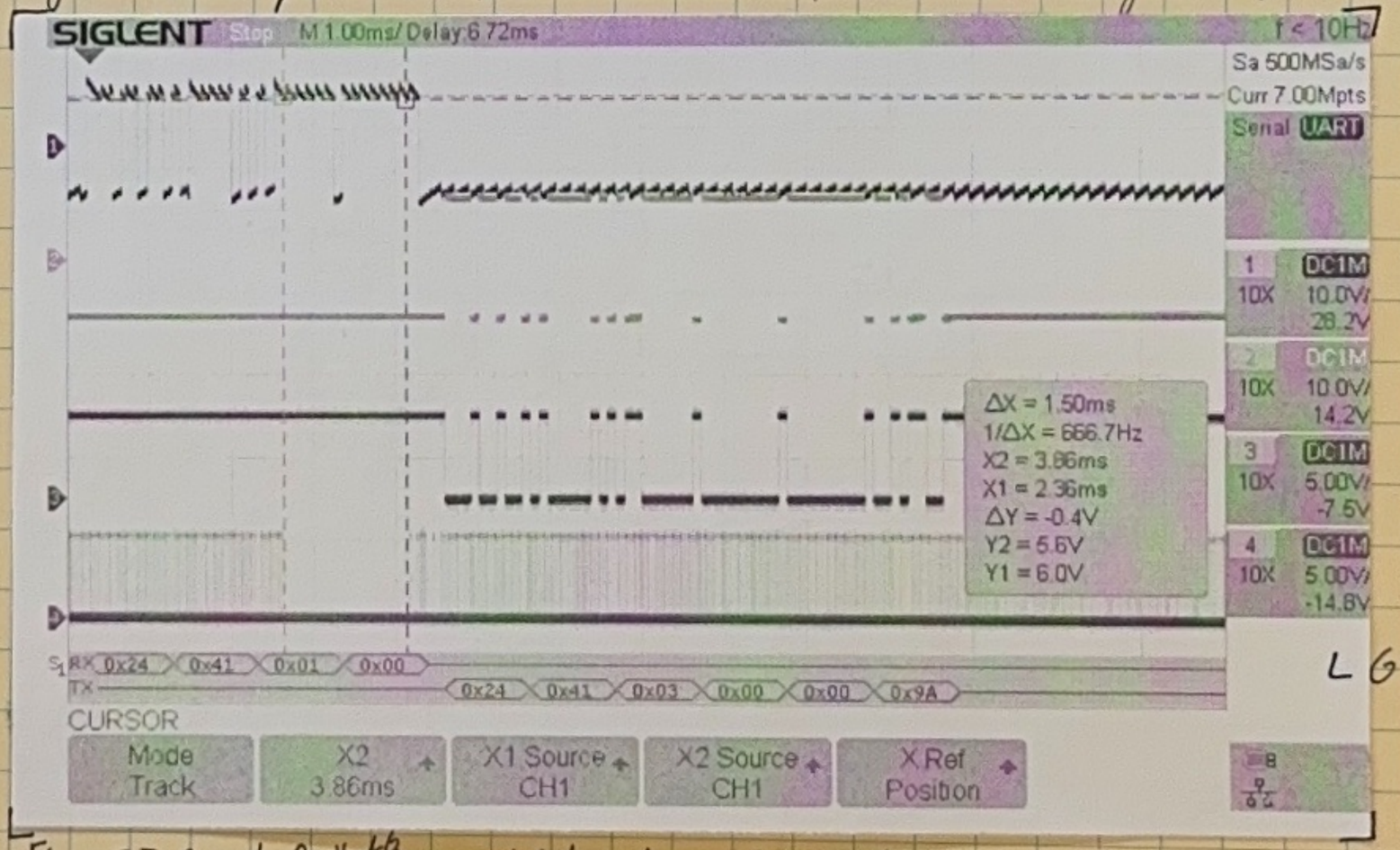
10/14/2026
LRG

Lab 8 - UART



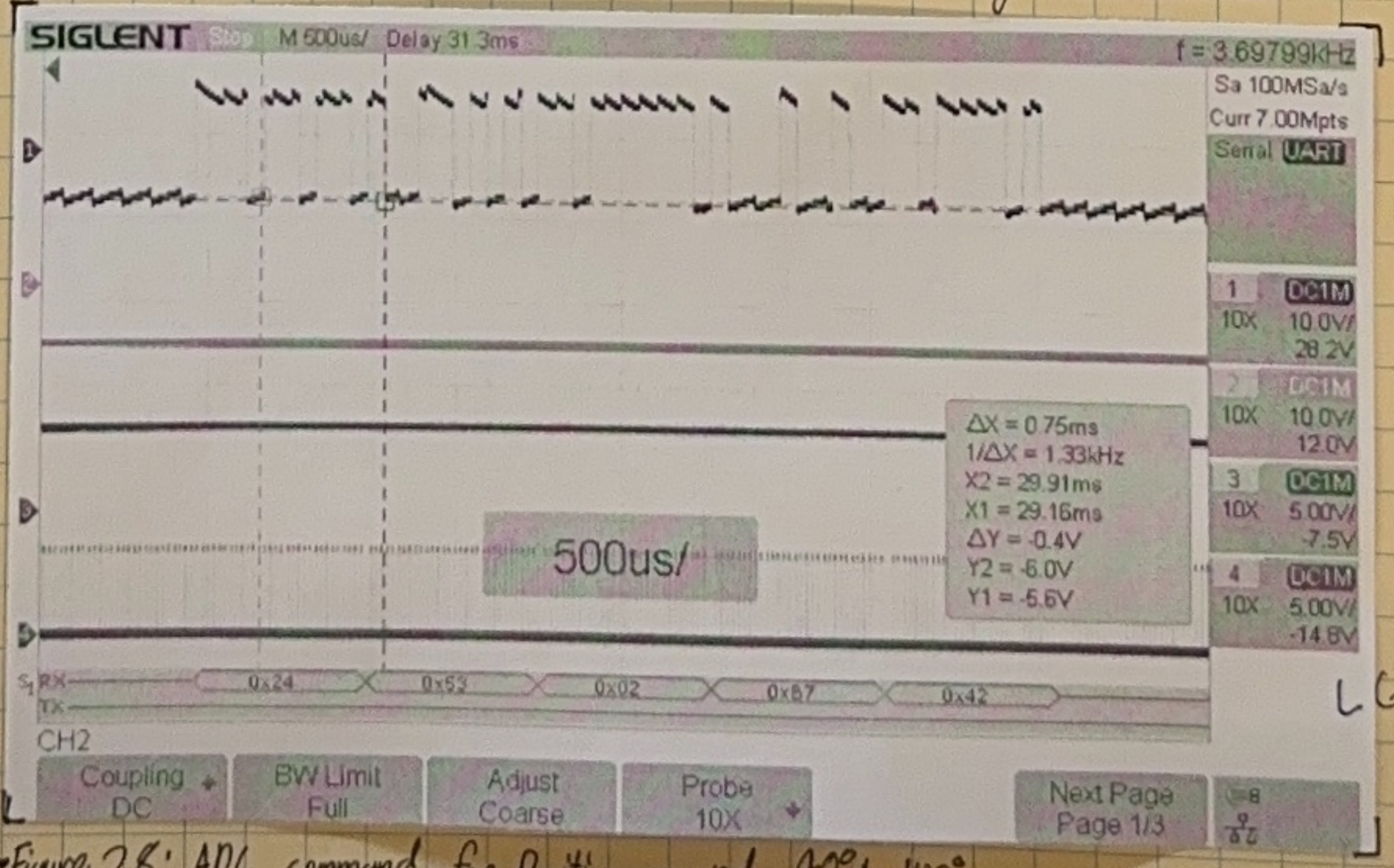
L6

•Figure 26: Request for temp and Returned temp value and message. (Temp 81 ish)



L6

•Figure 27: Request for ~~them~~ L6 temp and Return temp value and message (Temp 71 ish)



L6

10/A/2026
LRG

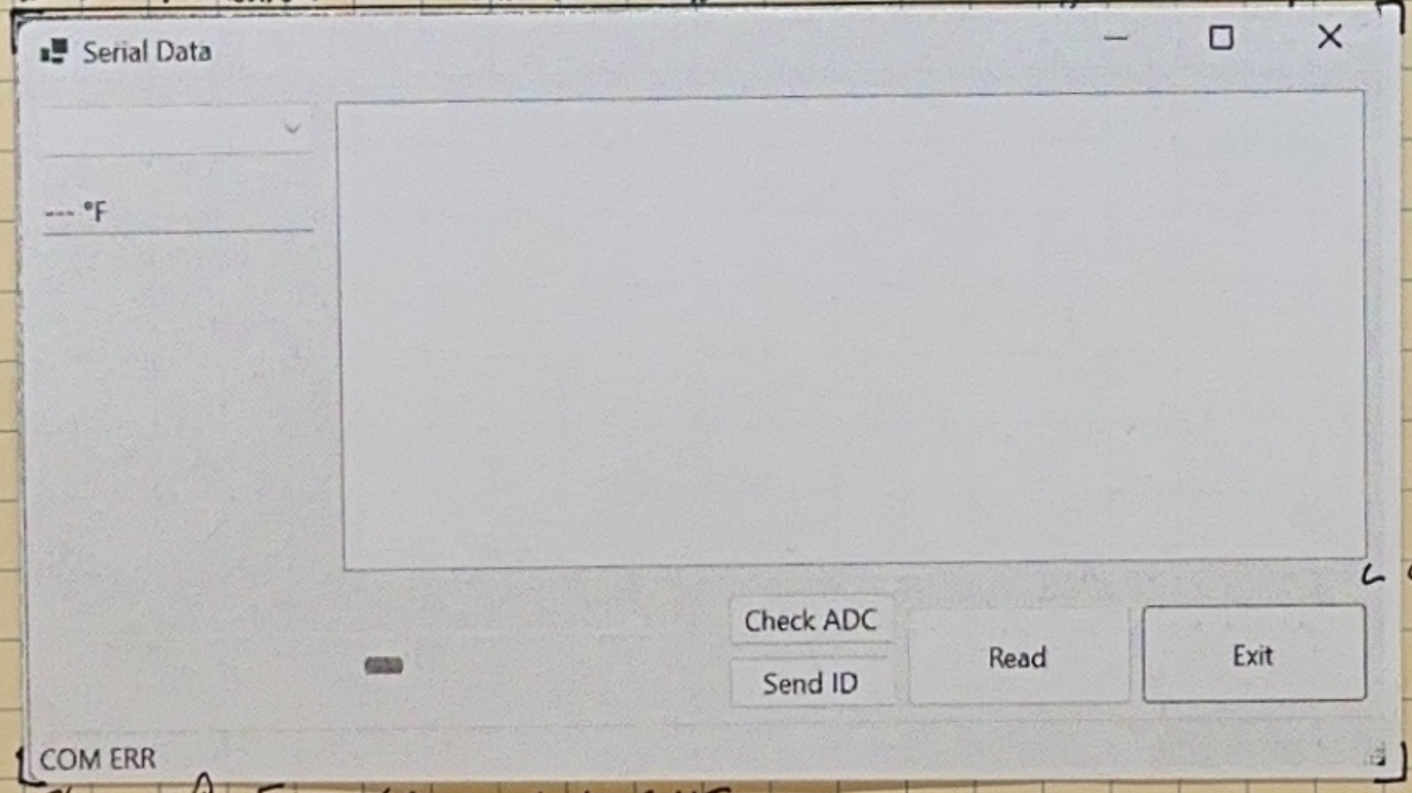
•Figure 28: ADC command for Position around 90° to 100°

Lab 8 - UART

10/19/2026
LRG

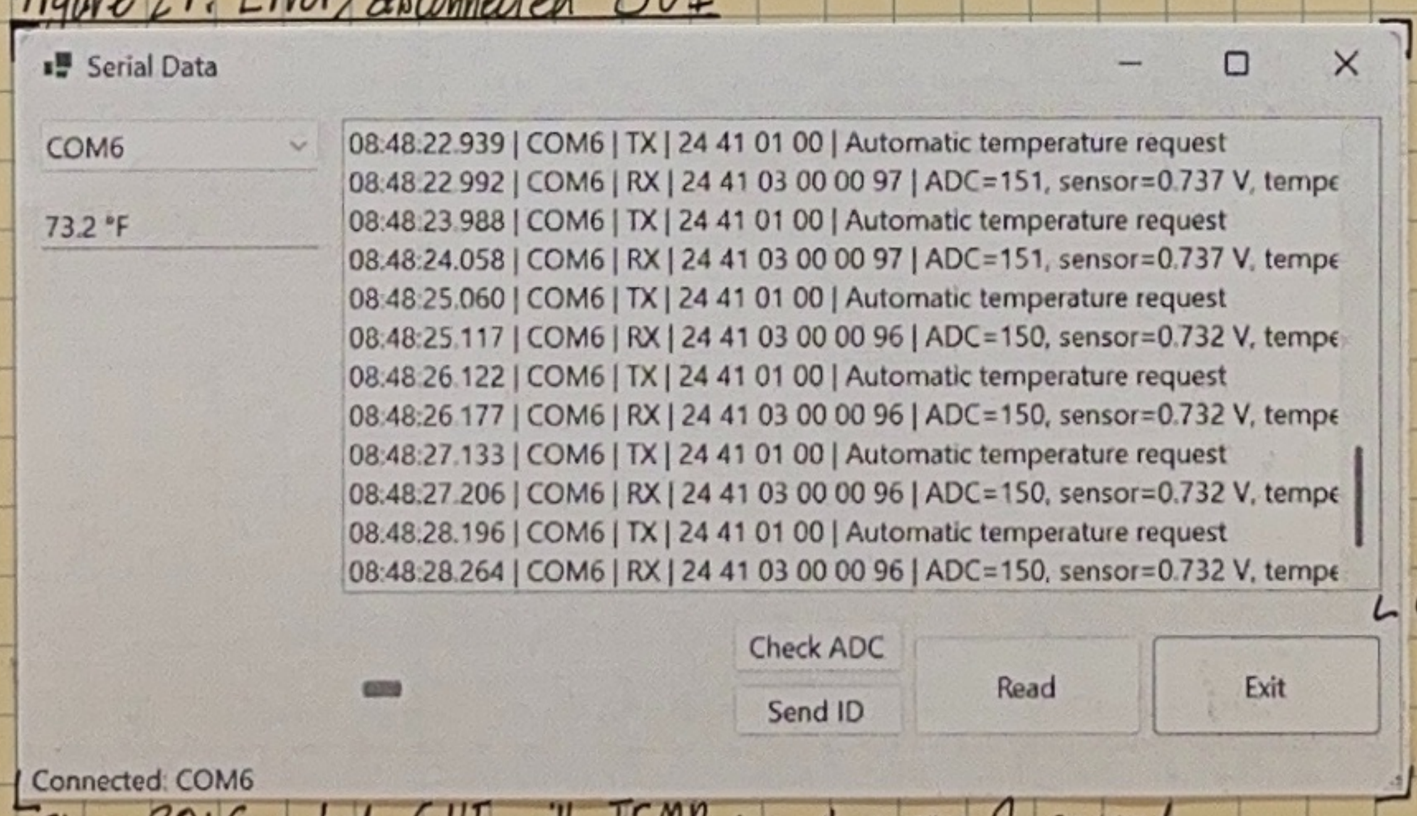
Measured VREF	ADC code	Measured LM34	Calculated LM34	Calculated °F	Reference °F
5v	~~140~~ LG	~~0.~~ LG not able to go that low	0.500 V	50°	– LG
5v	131	0.621	0.600 V	60°	–
5v	152	0.721	0.700 V	70°	71.9°F
5v	164	0.806	0.800 V	80°	–
5v	184	0.889	0.900 V	90°	–

Table 4: Measured and Calculated values for LM34 voltage and temp room temp 72°F



LG

Figure 29: Error/disconnected GUI



LG

TSE
10-19-26

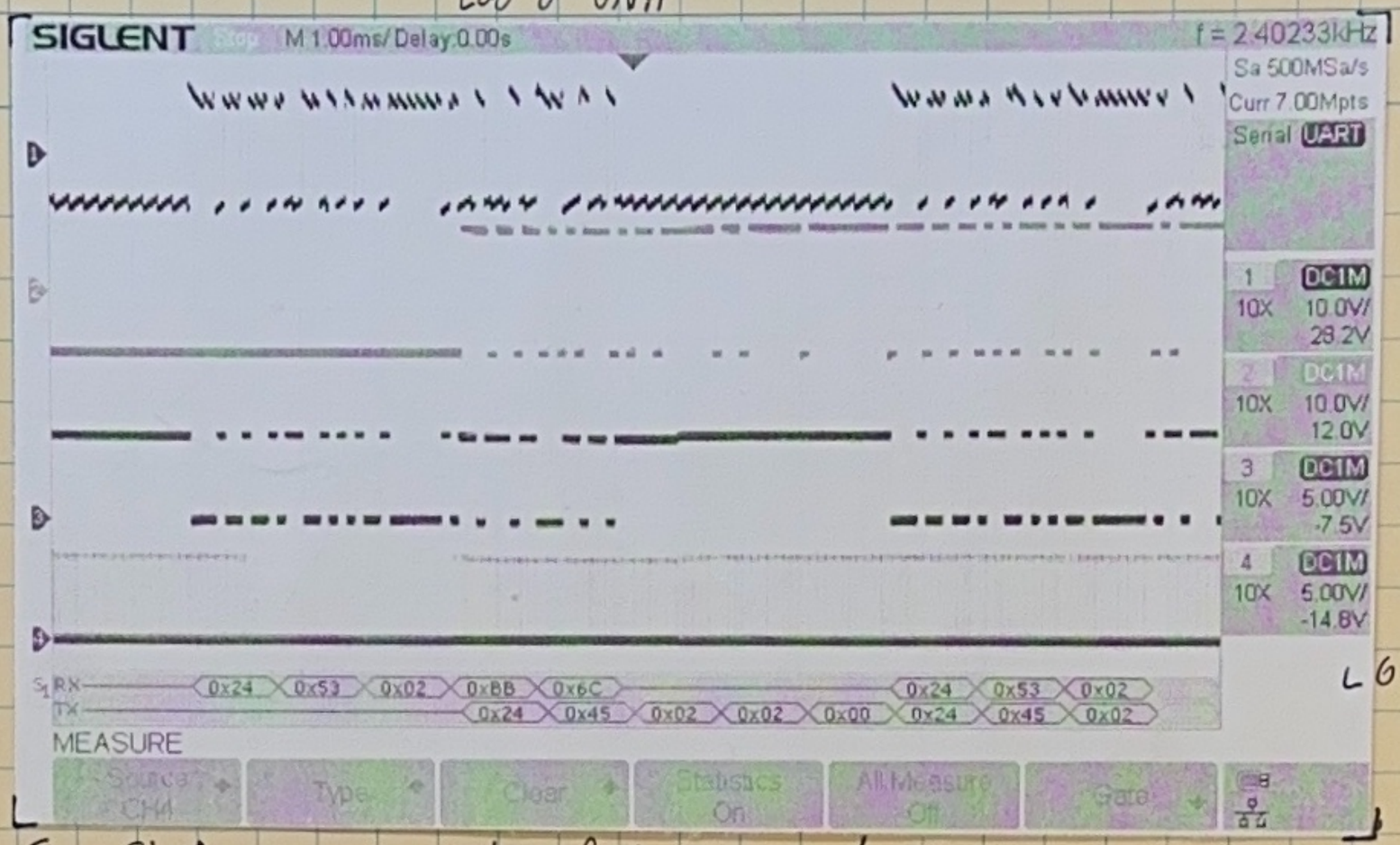
Figure 30: Connected GUI with TEMP request every 1 second.

-For the code for both Assembly and C# see the following git hub link- https://github.com/groeluke/Lab_8_UART.git (Serial data & Part 4).

-The LM34 conversion works by 10mV per, the PIC reports in a 10bit ADC count so the host has to convert ($V_{LM34} = ADC \times (VREF/1024)$) The ADC reference is VDD (5V) so that if ADC = 150 with VREF=5V gives 0.732v = 0.732v / 0.010V/°F = 73.2°F. The S packet and range validation. The packet is $, S, 2, MATCH_H, MATCH_L. The C# host converts the pulse width to the Timer2 compare value 0xB1E0 + PW, The parser holds the high byte until the low byte arrives. It checks if the value is in the valved range between 0xB3D4 and most 0xB0A4, comparing the high bytes first and low bytes only if the high bytes are equal. The biggest errors happen for the GUI with the disconnect not reconnecting.

10/19/2026
LRG

Lab 8 - UART



L6

•Figure 31: Overrun error capture of Servo Command over 2.5ms.

•Conclusion - Lab 8 covered (UART) Universal Asynchronous Receiver/Transmitter. We'r were asked to build a bidirectional serial interface for the PIC16F883, then integrate UART communication with ADC measurement and precise servo timing. Reusing working Lab 7 ADC configuration, acquisition, and result-reading code when integrating ADC in parts 3&4. We reused Lab 7 Timer1/CCP servo code and modifying it to work along side UART. Part 1 was ~~configuring~~ configuring the PIC to transmit data using UART at 9600 baud. Part 2 configured the PIC to receive serial data and send the same byte back to the computer. UART reception works and how to use interrupts and diagnostic signal monitor communication timing is the thing coverd in this section. Part 3 was creating packets to send commands and data between the computer and PIC. We used the ADC to read an analog input and return the measurement through UART. Part 4 was combing UART communication, a temp measurement, and comanding a servo. We used a LM34 to measure the temp in °F and displayed it on a C# windows form app and had a tracker bar to command a servo.

LG